REINFORCEMENT LEARNING

Sumário

REINFORCEMENT LEARNING 0

NOSSA HISTÓRIA 3

Aprendizagem por reforço 4

Conceitos básicos..... 5

Exploração x Intensificação 7

Política ótima 7

Métodos de aprendizado 12

Métodos de aproximação por função com atualização por gradiente 12

Características da Aprendizagem por Reforço 14

O Problema de Aprendizagem por Reforço 15

Fundamentos Matemáticos..... 19

Propriedade de Markov 19

Processos de Decisão Markoviano (PDM) 20

Métodos de Solução 24

Programação Dinâmica (PD) 24

Avaliação da Política 25

Avaliação da Politica Iterativa 26

Política Ótima 27

Monte Carlo (MC) 31

Avaliação da Política 32

Política Ótima 34

Diferença Temporal (DT)..... 35

Avaliação da Política - Predição DT 35

Vantagens dos Métodos de Predição DT 36

Q-learning 37

Considerações Finais 41

Utilização de Aprendizagem por Reforço para Modelagem Autônoma do Aprendiz em um Tutor Inteligente 42

INTRODUÇÃO 42
TÉCNICA DE APRENDIZADO POR REFORÇO 43
Q-Learning 45
ESTRUTURA DO SISTEMA 47
Definição do Conjunto dos Estados do Estudante (Perfil) 48
EXPERIMENTOS 49
Resultados Obtidos 50
CONCLUSÃO 54
Caracterizando o Sistema com uma análise das Vantagens e Desvantagens. 54
ESTUDO DE CASO II 55
Introdução 56
Processo Decisório de Markov 57
Definição MDP 57
Aprendizagem por Reforço 59
Modelagem do Problema 62
Resultados 64
Conclusões e Trabalhos Futuros 74
REFERÊNCIAS 76

NOSSA HISTÓRIA

A nossa história inicia-se com a ideia visionária e da realização do sonho de um grupo de empresários na busca de atender à crescente demanda de cursos de Graduação e Pós-Graduação. E assim foi criado o Instituto, como uma entidade capaz de oferecer serviços educacionais em nível superior.

O Instituto tem como objetivo formar cidadão nas diferentes áreas de conhecimento, aptos para a inserção em diversos setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e assim, colaborar na sua formação continuada. Também promover a divulgação de conhecimentos científicos, técnicos e culturais, que constituem patrimônio da humanidade, transmitindo e propagando os saberes através do ensino, utilizando-se de publicações e/ou outras normas de comunicação.

Tem como missão oferecer qualidade de ensino, conhecimento e cultura, de forma confiável e eficiente, para que o aluno tenha oportunidade de construir uma base profissional e ética, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido. E dessa forma, conquistar o espaço de uma das instituições modelo no país na oferta de cursos de qualidade.

Aprendizagem por reforço

Nas abordagens tradicionais de aprendizagem automática, geralmente os sistemas aprendem através de exemplos de pares de entrada e saída, que fornecem indicativos do comportamento esperado do sistema, tendo como tarefa aprender uma determinada função que poderia ter gerado tais pares. Estes métodos são apropriados quando existe alguma espécie de “professor” fornecendo os valores corretos ou quando a saída da função representa uma predição sobre o futuro que pode ser verificada pelas percepções do agente no próximo passo de iteração [38]. Mas, quando se deseja que o agente tenha uma total autonomia, este terá que ser capaz de aprender com base em outras informações, como por exemplo, recompensas ou reforços fornecidos por um “crítico” ou pelo próprio ambiente. Em certos casos é possível que o próprio agente determine as suas recompensas através da observação das transições de estado que realiza no ambiente, passando este a “experimentar” autonomamente o ambiente no qual está inserido [7].

Segundo Sutton e Barto em [41], Aprendizagem por Reforço é uma abordagem da Inteligência Artificial que permite a um indivíduo aprender a partir da sua interação com o ambiente onde ele se encontra, através do conhecimento sobre o estado do indivíduo no ambiente, das ações efetuadas no ambiente e das mudanças de estado que aconteceram depois de efetuadas as ações, que é um conceito básico na área de Aprendizado de Máquina. Aprendizagem por Reforço é antes de tudo indicado quando se deseja obter a política ótima (representa o comportamento que o agente segue para alcançar o objetivo) nos casos em que não se conhece a priori a função que modela esta política. O agente deve interagir com seu ambiente diretamente para obter informações, que serão processadas através de um algoritmo apropriado, afim de executar a ação que maximize a satisfação dos seus objetivos nos estados do ambiente.

Assim, AR consiste no aprendizado do mapeamento de estados em ações de modo que um valor numérico de retorno seja maximizado. A princípio, o

sistema não precisa conhecer as ações que deve tomar, mas deve descobrir quais ações o levam a obter maiores valores de retorno. Estes valores de retorno podem ser vistos como imediatos (locais) ou como retornos a longo prazo (globais), que neste último caso, permitem orientar o indivíduo a alcançar um dado objetivo. Neste capítulo abordaremos os conteúdos voltados à AR de forma mais ampla, apresentando suas características, seus problemas, sua formulação matemática e seus métodos de resolução, dando uma maior ênfase ao método conhecido como Diferença Temporal (DT), mais precisamente à resolução baseada no algoritmo Q-learning, que foi o algoritmo adotado nos experimentos do estudo de caso deste documento.

Aprendizado por reforço é um ramo estudado em estatística, psicologia, neurociência e ciência da computação. Atraiu o interesse de pesquisadores ligados a aprendizado de máquina e inteligência artificial, e é um método de programação de agentes através do oferecimento de recompensas e punições, sem a necessidade de especificar como uma tarefa deve ser realizada.

É entendido como o problema encontrado por um agente que deve aprender como se comportar em um ambiente dinâmico através de interações do tipo “tentativa e erro”. A abordagem que é utilizada nesse trabalho é feita usando-se técnicas estatísticas e métodos de programação dinâmica, buscando estimar qual a vantagem em se tomar determinadas ações em diferentes estados do ambiente. Uma boa referência no assunto é [KAE], e neste capítulo são apresentados os principais aspectos necessários para a compreensão da abordagem prática, a ser apresentada no capítulo seguinte.

Conceitos básicos

Em um ambiente de aprendizado por reforço, um agente está inserido em um ambiente T e interage com ele através de percepções e ações, conforme a Figura 1. A cada passo, o agente recebe como entrada e, uma indicação do estado (s) atual do ambiente. O agente escolhe, então, uma ação a a tomar, e gera sua saída. A ação altera então o estado do ambiente, e uma medida dessa mudança de estado é informada ao agente através de um valor de sinal de reforço, r. A função que mapeia os estados do ambiente nas ações que o agente

deve tomar é definida como a política do agente. A política de comportamento do agente, P, deve escolher tomar ações que maximizem o valor final da soma dos reforços recebidos em um intervalo de tempo. Tal política de comportamento pode ser aprendida através de um processo de tentativa e erro, guiado por diferentes algoritmos.

Figura 1 – Modelo padrão de aprendizado por reforço

Formalmente, o modelo é constituído por:

- Um conjunto discreto de estados que o ambiente pode assumir;
- Um conjunto discreto de ações que o agente pode tomar sobre o ambiente;
- Um conjunto de valores escalares de reforço; geralmente , ou os números reais.

Na Figura 1 existe ainda a função de entrada e, que é a maneira como o agente "lê" o estado atual do ambiente. A tarefa que o agente deve desempenhar é encontrar uma política π, que mapeia estados em ações, que maximiza a medida de reforço a longo prazo. Geralmente o sistema é não-determinístico; isso é, uma mesma ação tomada em um mesmo estado pode levar a diferentes estados, com diferentes valores de retorno percebidos.

Ao contrário dos métodos conhecidos de aprendizado supervisionado, no aprendizado por reforço não existem pares "entrada/saída", para serem

utilizados no treinamento. Após tomar uma ação, o agente imediatamente recebe uma recompensa, mas não fica sabendo qual deveria ser a melhor ação para atingir o objetivo (maximizar o retorno a longo prazo). Ele precisa obter experiência dos possíveis estados, ações, transições e recompensas do sistema para atingir a otimalidade.

Exploração x Intensificação

Um agente deve aprender, através de suas interações com o ambiente, qual a política que melhor o leva a atingir seu objetivo. Entretanto, como o agente não possui conhecimento sobre todas as variáveis envolvidas no processo, é necessário que ele explore, isso é, busque políticas alternativas àquelas que já utiliza e compare seus resultados com os já aprendidos até o momento. Digamos que, com o conhecimento adquirido até um determinado momento, o agente julgue que, para um dado estado , a melhor ação a ser tomada seja . Pode ser que exista uma ação alternativa que leve a um melhor resultado. Só será possível ao agente aprender essa nova informação se ele possuir um mecanismo de exploração que tome ações aleatórias com uma determinada taxa.

Essa taxa de exploração, se for muito alta, pode comprometer o alcance do objetivo, que é maximizar as recompensas. Um agente que decida explorar muito seu ambiente pode deixar de aproveitar o conhecimento já adquirido. Soluções híbridas são interessantes: no início do processo, quando o agente ainda possui poucas informações sobre o ambiente, a taxa de exploração mantémse alta; com o tempo essa taxa cai, levando a um maior aproveitamento do conhecimento adquirido e, consequentemente, maior probabilidade de obtenção de maiores retornos.

Política ótima

Em um problema qualquer de aprendizado por reforço, deseja-se que o agente aprenda uma política que seja eficaz, no sentido de obter, ao final da

execução de uma sequência (limitada ou não) de ações, o melhor valor possível como recompensa acumulada.

Quando o sistema encontra-se em determinado estado s, o valor recebido $V\pi(s)$ como recompensa, após a execução das ações seguindo uma política π, é definido como as soma das recompensas recebidas a cada ação tomada. Ações tomadas mais tarde podem ter peso menor, e isso é compensado com um fator de desconto temporal, γ.

$$V_\pi(s) = E\left(\sum_{t=0}^{\infty} \gamma^t r_t\right)$$

Para cada estado que o sistema assume, o valor máximo para a recompensa acumulada recebida após seguir-se a política ótima é definido como:

$$V^*(s) = \max_\pi E\left(V_\pi^*(s)\right)$$

Seja a recompensa percebida ao se tomar a ação a no estado s, a distribuição de probabilidade para, ao se tomar a ação a no estado s, o ambiente migrar para o estado s', o valor esperado para a soma das recompensas das ações tomadas seguindo a política ótima $\pi *$, a partir do estado s', γ um fator de desconto no tempo, e S o conjunto de estados que o ambiente pode assumir. Quando as recompensas recebidas em instantes diferentes de tempo apresentam a mesma importância para o sistema como um todo, o valor de γ fica fixado como 1.

Para encontrar o valor máximo de cada estado s, resolve-se iterativamente o sistema dado por:

$$V^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s' \in S} T(s,a,s')V^*(s')\right), \forall s \in S$$

Para exemplificar, consideremos o seguinte caso hipotético: um sistema pode assumir três estados distintos, s1, s2 e s3. A qualquer instante, duas ações

podem ser tomadas: a1 e a2. De acordo com o estado que o sistema se encontre, tomar uma ação pode levar a outro estado, com diferente distribuição de probabilidade. Por exemplo, podemos ter a seguinte distribuição de probabilidade:

Estado inicial s	Ação a	Novo estado s'	Probabilidade $T(s,a,s')$
s_1	a_1	s_1	0.1
s_1	a_1	s_2	0.7
s_1	a_1	s_3	0.2
s_1	a_2	s_1	0.2
s_1	a_2	s_2	0.5
s_1	a_2	s_3	0.3
s_2	a_1	s_1	0.3
s_2	a_1	s_2	0.3
s_2	a_1	s_3	0.4
s_2	a_2	s_1	0.1
s_2	a_2	s_2	0.3
s_2	a_2	s_3	0.6
s_3	a_1	s_1	0.8
s_3	a_1	s_2	0.1
s_3	a_1	s_3	0.1
s_3	a_2	s_1	0.1
s_3	a_2	s_2	0.8
s_3	a_2	s_3	0.1

Tabela 2 – Tabela de probabilidade de transição de estado

Da mesma forma, espera-se receber uma recompensa ao tomar-se uma ação em um determinado estado. A tabela 3 traz um exemplo

Estado	Ação	Recompensa
s_1	a_1	1
s_1	a_2	0.5
s_2	a_1	0
s_2	a_2	1
s_3	a_1	1
s_3	a_2	0.5

Tabela 3 – Exemplo de recompensas por ações tomadas em diferentes estados

A cada estado é também associado um valor que espera-se obter ao tomar as ações determinadas pela política à partir dele. Podemos considerar a seguinte tabela:

Estado	Valor esperado V(s)
s_1	10
s_2	15
s_3	18

Tabela 4 – Valores esperados ao seguir uma política à partir de diferentes estados.

Resolver, então, para V (s), requer que se resolva iterativamente um sistema de equações para cada estado, obtendo-se assim valores atualizados a cada iteração, e utilizando-os na estimativa seguinte. Vejamos como seria resolvido para s1, na primeira iteração. Da equação 10, temos que:

$$V^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V^*(s') \right), \forall s \in S$$

$$V^*(s_1) = \max_a \begin{pmatrix} R(s_1,a_1) + \gamma\big(T(s_1,a_1,s_1)V^*(s_1) + T(s_1,a_1,s_2)V^*(s_2) + T(s_1,a_1,s_3)V^*(s_3)\big), \\ R(s_1,a_2) + \gamma\big(T(s_1,a_2,s_1)V^*(s_1) + T(s_1,a_2,s_2)V^*(s_2) + T(s_1,a_2,s_3)V^*(s_3)\big) \end{pmatrix}$$

Utilizando os valores das tabelas 2, 3 e 4, temos:

$$V^*(s_1) = \max_a \begin{pmatrix} 1.0 + \gamma(0.1 * 10 + 0.7 * 15 + 0.2 * 18), \\ 0.5 + \gamma(0.2 * 10 + 0.5 * 15 + 0.3 * 18) \end{pmatrix}$$

Considerando $\gamma = 1$, para os valores de ações tomadas no futuro tendo o mesmo peso das ações tomadas no presente, temos:

$$V^*(s_1) = \max_a \begin{pmatrix} 1.0 + (1 + 10.5 + 3.6), \\ 0.5 + (2 + 7.5 + 5.4) \end{pmatrix}$$

$$V^*(s_1) = \max_a \begin{pmatrix} 16.1, \\ 15.4 \end{pmatrix}$$

O valor V (s1) de passa a ser, então, 16.1, que deve ser alcançado tomando-se a ação a1. Deve-se repetir o mesmo procedimento para V (s2) e V (s3) . Após obter-se os valores esperados, repete-se a resolução do sistema, até que haja convergência dos valores.

Sabe-se que a política ótima , definida como aquela que maximiza o retorno esperado das recompensas futuras, ao ser seguida à partir do estado s, respeita a equação:

$$\pi^*(s) = arg \max_a \left(R(s,a) + \gamma \sum_{s' \in S} T(s,a,s')V^*(s') \right)$$

No modelo geral de aprendizado por reforço, as ações determinam não apenas a recompensa imediatamente recebida pelo agente, mas também qual será o próximo estado do ambiente (se não deterministicamente, ao menos probabilisticamente). Assim, para um agente escolher qual ação tomar, não é apenas o valor da recompensa que deve ser considerado, mas também o próximo estado a ser atingido, e o valor esperado da recompensa a longo prazo

que será obtida tomando-se esse caminho. Costuma-se chamar de experimento uma sequência de ações tomadas por um agente, desde um estado inicial (aleatório ou não), seguindo uma política, até um estado final predeterminado. Ao final do experimento, obtém-se o valor da recompensa total como a somatória das recompensas obtidas a cada ação realizada.

Métodos de aprendizado

Existem problemas nos quais a quantidade de estados que podem ser assumidos pelo ambiente e também a quantidade de ações que o agente pode tomar são computacionalmente tratáveis. Nesses casos, é possível realizar o treinamento armazenando-se todo o conjunto de estados, ações e tabelas de transições em memória, definir as recompensas para as ações e iniciar o treinamento, colocando o agente para interagir com o ambiente.

Os métodos que trabalham dessa forma diferem-se na maneira que realizam o aprendizado e a atualização da política. No método Monte Carlo, por exemplo, as ações são tomadas, as tabelas de mapeamento são atualizadas, as recompensas acumuladas. Após a realização de todo um experimento (início em um estado qualquer e término do sistema, no estado final), o valor da recompensa acumulada obtida é utilizado para reforçar as ações tomadas nesse experimento, positiva ou negativamente. Já no método Q-Learning, bastante semelhante, a cada ação tomada pelo agente o valor da recompensa obtida associado ao valor esperado para o estado alcançado são utilizados para reforçar a ação recém tomada. Não é possível apontar qual desses métodos é melhor; em um problema o resultado alcançado por um método é melhor, em outro problema, a situação se inverte.

Métodos de aproximação por função com atualização por gradiente

Quando o espaço de estados ou de ações é muito grande, fica complicado manter uma tabela com todos os valores esperados para os pares estado x ação. Para resolver esse problema, a tabela é aproximada por uma função que simula seu comportamento. Assim, para obter o valor ótimo esperado para um

determinado estado ( ), ao invés de recorrer à tabela realizamos uma operação em uma função, que recebe como entrada um conjunto de características do estado e retorna o valor ótimo para esse estado.

Essa função de aproximação é construída, como foi dito, sobre um conjunto de características do estado. Dessa forma, estados “parecidos” são agrupados pela própria modelagem da função, e o tratamento do grande conjunto de estados passa a ser computacionalmente viável. Determinar quais são as características do estado a serem consideradas na função faz parte da modelagem do problema, e não é fácil acertar na sua escolha. Essas características são combinadas (linearmente ou não) na função, e o aprendizado se dá na determinação e atualização dos coeficientes que ponderam, na função, as características do estado.

No treinamento desse método, os coeficientes reais da função são agrupados em um vetor coluna:

a $\overrightarrow{\theta_t} = \left(\theta_t(1), \theta_t(2), \dots, \theta_t(n)\right)^T$

(T denota “transposta”), e deve ser uma função suave diferenciável de para todo . Assume-se que a cada passo t do algoritmo, um novo estado é observado. Mesmo que o valor obtido seja exato, o problema ainda é complicado porque a função de aproximação possui recursos e resolução limitados. Não há conjunto que contemple todos os estados, nem mesmo os exemplos varridos no treinamento, corretamente. Ainda mais porque a função deve generalizar os estados que não apareceram no conjunto de exemplos.

A referência [SUT] apresenta como minimizar o erro quadrático médio da função de aproximação minimizando o erro no conjunto de exemplos. A cada novo exemplo obtido no algoritmo ajusta-se o vetor de parâmetros com uma pequena variação na direção que melhor reduz o erro obtido no exemplo (o gradiente da função de aproximação, com as derivadas parciais nos coeficientes do vetor . Esse vetor de derivadas é, então, o gradiente de f em relação a . O método é chamado de “atualização por gradiente” porque a variação de é

proporcional ao gradiente negativo do erro quadrático do exemplo verificado, que é a direção na qual o erro diminui mais rapidamente.

Características da Aprendizagem por Reforço

Aprendizado pela Interação: essa é a característica principal que define um problema de Aprendizagem por Reforço. Onde um agente AR age no ambiente e aguarda pelo valor de reforço que o ambiente deve informar como resposta perante a ação tomada, assimilando através do aprendizado o valor de reforço obtido para tomar decisões posteriores.

Retorno Atrasado: um máximo valor de reforço que o ambiente envia para o agente não quer dizer necessariamente que a ação tomada pelo agente foi a melhor. Uma ação é produto de uma decisão local no ambiente, sendo seu efeito imediato de natureza local, enquanto, em um sistema de Aprendizagem por Reforço, busca-se alcançar objetivos globais no ambiente. Assim as ações tomadas devem levar a maximizar o retorno total, isto é, a qualidade das ações tomadas é vista pelas soluções encontradas à longo prazo.

Orientado ao Objeto: em Aprendizagem por Reforço, o problema tratado é considerado como um ambiente que dá respostas perante ações efetuadas, não sendo necessário conhecer detalhes da modelagem desse ambiente. Simplesmente, existe um agente que age dentro do ambiente desconhecido tentando alcançar um objetivo. O objetivo é, geralmente, otimizar algum comportamento dentro do ambiente.

Investigação x Exploração: Em aprendizagem por reforço os agentes vivem um dilema conhecido na literatura como “the Exploration × Exploitation dilemma”, que consiste em decidir quando se deve aprender e quando não se deve aprender sobre o ambiente, mas usar a informação já obtida até o momento. Para que um sistema seja realmente autônomo, esta decisão deve ser tomada pelo próprio sistema.

A decisão é fundamentalmente uma escolha entre agir baseado na melhor informação de que o agente dispõe no momento ou agir para obter novas informações sobre o ambiente que possam permitir níveis de desempenho

maiores no futuro. Isto significa que o agente deve aprender quais ações maximizam os valores dos ganhos obtidos no tempo, mas também, deve agir de forma a atingir esta maximização, explorando ações ainda não executadas ou regiões pouco visitadas do espaço de estados. Como ambas as formas trazem, em momentos específicos, benefícios à solução dos problemas, uma boa estratégia é mesclar as formas.

Este é um problema crucial no contexto da aprendizagem por reforço, pois agir para obter informação pode aumentar o desempenho à longo prazo, embora faça com que o desempenho a curto prazo diminua. Tomando-se estes cuidados, quanto mais tempo o agente estiver atuando no ambiente, mais corretas serão suas ações no decorrer de sua tarefa.

O Problema de Aprendizagem por Reforço

Um sistema típico de Aprendizagem por Reforço constitui-se basicamente de um agente interagindo em um ambiente via percepção e ação. Ou seja, o agente percebe as situações dadas no ambiente, pelo menos parcialmente, e baseado nessas medições, seleciona uma ação a tomar no ambiente. A ação tomada muda de alguma forma o ambiente, afetando o estado na tentativa de alcançar o objetivo relacionado, e as mudanças são comunicadas ao agente através de um sinal de reforço. Como pode ser visto na Figura 4.1 (traduzida a partir da original, obtida no livro de Sutton e Barto [41]).

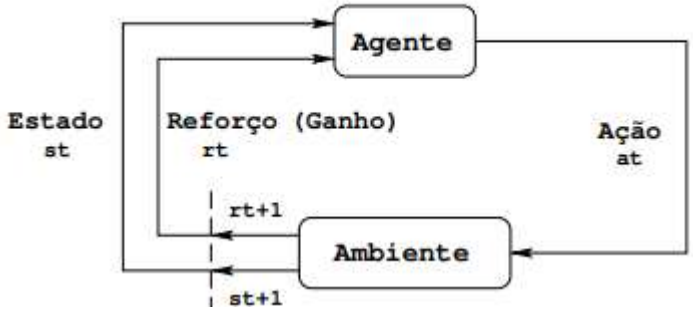


Figura 1: A interação agente-ambiente na Aprendizagem por Reforço

Os efeitos das ações não podem ser perfeitamente antecipados, com isso, o agente deve monitorar o ambiente freqüentemente e reagir apropriadamente.

Em um sistema de AR, o estado do ambiente é representado por: 1) um conjunto de variáveis de estado percebidas pelo agente, onde o conjunto das combinações de valores dessas variáveis forma o conjunto de estados discretos do agente (S); 2) um conjunto de ações discretas, que escolhidas por um agente muda o estado do ambiente (A(s)) e 3) valor das transições de estados, que é passado ao agente através de um sinal de reforço, denominado ganho (valores tipicamente entre [0,1]).

O objetivo do método é levar o agente a escolher a seqüência de ações que tendem a aumentar a soma de valores de reforço, ou seja, é encontrar a política π, definida como o mapeamento de estados em ações que maximize as medidas do reforço acumuladas no tempo. O Problema de Aprendizagem por Reforço apresenta cinco partes fundamentais, que são:

O Ambiente: Todo sistema de AR aprende um mapeamento de situações e ações por experimentação em um ambiente dinâmico. O ambiente no qual está inserido o sistema, deve ser pelo menos parcialmente observável através de sensores, descrições simbólicas, ou situações mentais. Também é possível, entretanto, que toda informação relevante do ambiente esteja perfeitamente disponível. Neste caso, o agente poderá escolher ações baseadas em estados reais do ambiente.

. A Política: Uma política expressa pelo termo π, representa o comportamento que o sistema AR segue para alcançar o objetivo. Em outras palavras, uma política π é um mapeamento de estados s e ações a em um valor π(s,a). Assim, se um agente AR muda a sua política, então as probabilidades de seleção de ações sofrem mudanças e conseqüentemente, o comportamento do sistema apresenta variações à medida que o agente vai acumulando experiência a partir das interações com o ambiente. Portanto, o processo de aprendizado no sistema AR pode ser expresso em termos da convergência até uma política ótima ($\pi * (s,a)$) que conduza à solução do problema de forma ótima.

Reforço e Retorno: O Reforço é um sinal do tipo escalar (rt+1), que é devolvido pelo ambiente ao agente assim que uma ação tenha sido efetuada e uma transição de estado (st → st+1) tenha ocorrida. Existem diferentes formas

de definir o reforço para cada transição no ambiente, gerando-se funções de reforço que, intrinsecamente, expressam o objetivo que o sistema AR deve alcançar. O agente deve maximizar a quantidade total de reforços recebidos chamado de retorno, que nem sempre significa maximizar o reforço imediato a receber, mas o reforço acumulado durante a execução total.

De modo geral, o sistema AR busca maximizar o valor esperado de retorno, com isso, o retorno pode ser definido como uma função da seqüência de valores de reforço até um tempo T final. No caso mais simples é um somatório como aparece na equação seguinte:

$$R_T = r_{t+1} + r_{t+2} + r_{t+3} + \ldots + r_T$$

Em muitos casos a interação entre agente e ambiente não termina naturalmente em um episódio (seqüência de estados que chegam até o estado final), mas continua sem limite, como por exemplo em tarefas de controle contínuo. Para essas tarefas a formulação do retorno é um problema, pois T = ∞ e o retorno que se deseja também tenderá ao infinito (RT = ∞). Para estes problemas foi criada a taxa de amortização (γ), a qual determina o grau de influência que têm os valores futuros sobre o reforço total. Assim, a expressão do retorno aplicando taxa de amortização é expressa pela seguinte equação:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

onde, $0 \leq \gamma \leq 1$, se $\gamma \rightarrow 0$, o agente tem uma visão míope dos reforços, maximizando apenas os reforços imediatos, e se $\gamma \rightarrow 1$, a visão do reforço abrange todos os estados futuros dando maior importância ao estado final, desde que a seqüência RT seja limitada. Um sistema de AR faz um mapeamento de estados em ações baseado nos reforços recebidos. Assim, o objetivo do AR é definido usando-se o conceito de função de reforço, a qual é uma função dos reforços futuros que o agente procura maximizar. Ao maximizar essa função, o objetivo será alcançado de forma ótima. A função de reforço define quais são os bons e maus eventos para os agentes.

Função de Reforço: As funções de reforço podem ser bastante complicadas, porém existem pelo menos três classes de problemas freqüentemente usadas para criar funções adequadas a cada tipo de problema:

• Reforço só no estado final: Nesta classe de funções, as recompensas são todas zero, exceto no estado final, em que o agente recebe uma recompensa real (ex: +1) ou uma penalidade (ex: −1). Como o objetivo é maximizar o reforço, o agente irá aprender que os estados correspondentes a uma recompensa são bons e os que levaram a uma penalidade devem ser evitados.

• Tempo mínimo ao objetivo: Funções de reforço nesta classe fazem com que o agente realize ações que produzam o caminho ou trajetória mais curta para um estado objetivo. Toda ação tem penalidade (−1), sendo que o estado final é (0). Como o agente tenta maximizar valores de reforço, ele aprende a escolher ações que minimizam o tempo que leva a alcançar o estado final.

• Minimizar reforços: Nem sempre o agente precisa ou deve tentar maximizar a função de reforço, podendo também aprender a minimizá-las. Isto é útil quando o reforço é uma função para recursos limitados e o agente deve aprender a conservá-los ao mesmo tempo em que alcança o objetivo.

Função Valor-Estado: Define-se uma função valor-estado como o mapeamento do estado, ou par estado-ação em um valor que é obtido a partir do reforço atual e dos reforços futuros. Se a função valor-estado considera só o estado s é denotada como V(s), enquanto se é considerado o par estado-ação (s,a), então a função valor-estado é denotada como função valor-ação Q(s,a).

(a) Função valor-estado: Uma vez que os reforços futuros mantêm dependências das ações futuras, as funções valor dependem também da política π que o AR segue. Em um Processo de Decisão Markoviano (ver subseção 4.4.2) se define uma função valor-estado V π (s) dependente da política π como a equação:

$$V^{\pi}(s) = E_{\pi}\{R_t \mid s_t = s\} = E_{\pi}\{\sum_{k=o}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s\}$$

onde a função V π (s) é o valor esperado do retorno para o estado st = s. Isto é, o somatório dos reforços aplicando a taxa de amortização γ.

(b) Função valor-ação: Se consideramos o par estado-ação, a equação para a função valorestado Q π (s,a) será a seguinte:

$$Q^{\pi}(s,a) = E_{\pi}\{R_t \mid s_t = s, a_t = a\} = E_{\pi}\{\sum_{k=o}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a\}$$

que é semelhante à equação (4.3), só que considerando o reforço esperado para um estado st = s e uma ação at = a.

As equações apresentam as funções valor-estado e valor-ação respectivamente, que dependem exatamente dos valores de reforço, o qual implica o conhecimento completo da dinâmica do ambiente como um Processo de Decisão Markoviano.

Fundamentos Matemáticos

Existem dois conceitos que devem ser conhecidos para facilitar a modelagem de um problema como um sistema de Aprendizagem por Reforço. A seguir, apresentamos uma breve descrição destes conceitos.

Propriedade de Markov

Quando a probabilidade de transição de um estado s para um estado s 0 depende apenas do estado s e da ação a adotada em s, isso significa que o estado corrente fornece informação suficiente para o sistema de aprendizado decidir que ação deve ser tomada. Quando o sistema possui essa característica, diz-se que ele satisfaz a Propriedade de Markov [1]. No caso mais geral, se a resposta em t +1 para uma ação efetuada em t depende de todo o histórico de ações até o momento atual, a dinâmica do ambiente é definida pela especificação completa da distribuição de probabilidades, como mostra a equação abaixo:

$$Pr\{s_{t+1} = s', r_{t+1} = r \mid s_t,\ a_t,\ r_t,\ s_{t-1},\ a_{t-1}, \ldots,\ r_1,\ s_0,\ a_0\}$$

onde a probabilidade (Pr) do estado st+1 ser o estado s 0 e o reforço rt+1 ser igual a r é uma função que depende de todos os estados, ações e reforços passados. Se a resposta do ambiente em t +1 depende apenas dos estados e reforços em t, então, a probabilidade da transição para o estado s 0 é dada pela expressão da equação abaixo.

$$P^{a}_{s,s'} = Pr\{s_{t+1} = s' \mid s_t = s, a_t = a\}$$

A probabilidade de transição satisfaz às seguintes condições:

$$P^{a}_{s,s'} \geq 0, \forall s, s' \in S, \forall a \in A(s)$$

$$\sum_{s' \in S} P^{a}_{s,s'} = 1, \forall s \in S, \forall a \in A(s)$$

A Propriedade de Markov é de fundamental importância na AR, uma vez que tanto as decisões como os valores são funções apenas do estado atual, abrindo a possibilidade de métodos de soluções incrementais, onde pode-se obter soluções a partir do estado atual e para cada um dos estados futuros, como é feito no método de Programação Dinâmica.

Processos de Decisão Markoviano (PDM)

Segundo Bellman em [1], um Processo de Decisão Markoviano é definido como um conjunto de estados S, ∀s ∈ S, um conjunto de ações A(s), um conjunto de transições entre estados associadas com as ações e um conjunto de probabilidades P sobre o conjunto S que representa uma modelagem das transições entre os estados. Assim, dado um par de estado e ação, a probabilidade do estado s passar a um estado s 0 é: probabilidade do estado s passar a um estado s 0 é:

$$P^{a}_{s,s'} = Pr\{s_{t+1} = s' \mid s_t = s, a_t = a\}$$

onde Pr é o operador de probabilidade; neste caso representa-se a probabilidade do estado st+1 ser s 0 , sempre que o estado st for igual a s e a ação at for igual a a. Desta forma, a dependência que o estado seguinte st+1 seja o estado s 0

está relacionada a tomar a ação a no instante t. De forma análoga, dados um estado e ação atuais e um estado seguinte s 0 , o valor esperado do retorno é:

$$R^{a}_{s,s'} = E\{r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'\}$$

onde E é o valor esperado do retorno rt+1, sempre que o estado st no instante t passe a ser o estado s 0 no instante t +1.

Os valores de probabilidade P a ss0 e retorno esperado R a ss0 determinam os aspectos mais importantes da dinâmica de um PDM finito. Podemos caracteriza-lo como: 1) um ambiente evolui probabilisticamente baseado num conjunto finito e discreto de estados; 2) para cada estado do ambiente, existe um conjunto finito de ações possíveis; 3) cada passo que o sistema de aprendizado executar uma ação, é verificado um custo positivo ou negativo para o ambiente em relação à ação; e 4) estados são observados, ações são executadas e reforços são relacionados.

Assim para quase todos os problemas de Aprendizagem por Reforço é suposto que o ambiente tenha a forma de um Processo de Decisão Markoviano, desde que seja satisfeita a Propriedade de Markov no ambiente. Nem todos os algoritmos de AR necessitam uma modelagem PDM inteira do ambiente, mas é necessário ter-se pelo menos a visão do ambiente como um conjunto de estados e ações [41].

Exemplo:

Podemos demonstrar os conceitos apresentados até o momento através de um exemplo simples de um robô reciclador, não particularmente realístico, apresentado na seção 3.6 do livro do Sutton e Barto [41].

O robô funciona a bateria e tem como objetivo coletar o maior número de latas possíveis, gastando o mínimo de energia. Suas ações são baseadas por um agente AR, onde este decide se o robô irá: 1) procurar ativamente por latas por um determinado período de tempo, 2) permanecer parado a espera que alguém lhe traga as latas, ou 3) voltar para a base para recarregar as suas baterias. Essa decisão tem que ser tomada periodicamente ou sempre que determinados eventos ocorram, como encontrar uma lata vazia.

Na Figura 4.2, podemos ver caracterizado o problema de aprendizagem, onde temos um sinal que representa as escolhas feitas pelo agente (ação), um sinal que indica o estado do ambiente (estado) e outro que define as metas a serem alcançadas (ganhos).

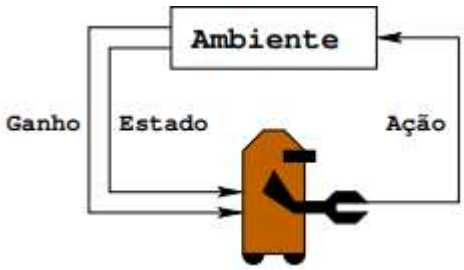


Figura 4.2: Modelo padrão de aprendizagem do robô reciclador

O agente toma suas decisões com base no nível de energia da bateria, onde podemos distinguir dois níveis, representados no livro [41] por (high, low), de modo que o espaço de estados é dado por S = {high, low}, e suas possíveis ações são conhecidas como (search, wait, recharge). Sabendo que o agente se baseia no nível de energia para tomar suas ações, quando este encontrar-se no nível high, tomar a ação recharge não seria sensata, assim não incluímos essa ação para este estado e o conjunto de ações do agente então passa a ser: A(high) = {search, wait} e A(low) = {search, wait, recharge}.

A cada lata coletada é adicionado uma recompensa +1 e caso ele fique sem carga o mesmo leva uma punição (−3). Como queremos que o robô colete o maior número possível de latas adotou-se um retorno R search ≥ R wait , visto que terá um melhor resultado, e adotou-se que nenhuma lata poderá ser coletada durante os períodos de recarga e quando a bateria estiver esgotada. Por este ser um sistema PDM finito, podemos apresentar as probabilidades de transição e os ganhos previstos, como na Tabela 5.2 (retirada do livro [41]).

$s = s_t$	$s' = s_{t+1}$	$a = a_t$	$P^a_{s,s'}$	$R^a_{s,s'}$
high	high	search	α	R^{search}
high	low	search	$1-\alpha$	R^{search}
low	high	search	$1-\beta$	-3
low	low	search	β	R^{search}
high	high	wait	1	R^{wait}
high	low	wait	0	R^{wait}
low	high	wait	0	R^{wait}
low	low	wait	1	R^{wait}
low	high	recharge	1	0
low	low	recharge	0	0

Tabela 4.1: Tabela das probabilidades das transições e retornos previstas para o PDM finito

Estando a bateria no nível high e executando a ação search tem-se duas possibilidades: uma com a probabilidade (α) se a bateria continuar no mesmo nível, e outra com uma probabilidade (1−α) se mudar para o nível low. Caso esteja no nível low e execute a ação search tem-se duas possibilidades: uma com probabilidade (β) de continuar no mesmo nível, e uma probabilidade (1 − β) de esgotar a bateria, neste caso o robô terá que ser salvo para recarregar a sua bateria. Pelo objetivo proposto o robô não deve ficar sem energia e caso isso ocorra ele é severamente punido.

Quando se escolhe a opção wait não há gasto de energia, ficando o robô no mesmo estado, desta forma aquelas opções em que há mudanças de estado têm probabilidade 0 de ocorrer. No caso de escolha da ação recharge o próximo estado será de bateria high, não havendo outra possibilidade.

Outra forma de representar a dinâmica de um PDM finito é através de um diagrama de transição de estados visto na Figura 4.3 (retirada do livro [41]), onde este possui um nó para cada estado possível (grande círculo marcando o nome do estado), e um nó para cada par estado-ação (círculo pequeno cheio), as suas setas representam as probabilidades (P a ss0) de executar a transição do estado s para s 0 se o agente tomar a decisão a e o retorno (R a ss0) que é obtido se executar a transição de s para s 0 após tomar a ação a.

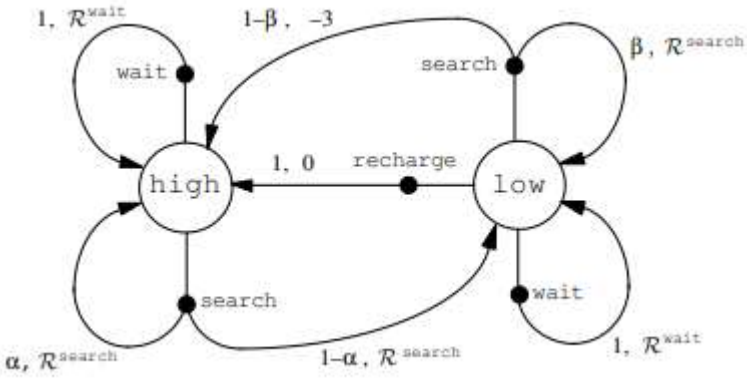


Figura 4.3: Gráfico das transições do robô reciclado

Métodos de Solução

Para solucionar o problema de Aprendizagem por Reforço, que são: 1) avaliação de política; e 2) encontrar a política ótima, existem três classes de métodos fundamentais, que apresentaremos e analisaremos nas subseções a seguir. Também ilustraremos estes métodos com o exemplo do robô reciclador, visto anteriormente.

Programação Dinâmica (PD)

Segundo Bellman em [1], a Programação Dinâmica tem a vantagem de ser matematicamente bem fundamentada, mas exige uma modelagem bem precisa do ambiente como um Processo de Decisão Markoviano.

Programação Dinâmica é uma coleção de algoritmos que podem obter políticas ótimas sempre que exista uma modelagem perfeita do ambiente como um PDM, isto é, como um conjunto de estados, ações, retornos e probabilidades da transição em todos os estados. Os algoritmos clássicos de PD são usados de forma limitada, uma vez que a modelagem perfeita do ambiente como PDM exige um grande custo computacional, porém, fornece um bom fundamento para o conhecimento dos outros métodos usados na solução do problema de AR e um padrão de comparação.

A dinâmica do sistema é dada por um conjunto de probabilidades de transição de estado, P a ss0 = Pr{st+1 = s 0 , st = s, at+1 = a}, e por um conjunto

de reforços imediatos esperados, R a ss0 = E{rt+1 | at = a, st = s, st+1 = s 0}, para todo s,s 0 ∈ S, a ∈ A(s).

Avaliação da Política

Segundo Monteiro em [26], as escolhas das ações do agente são feitas a partir de uma função do estado, chamada política (π : S → A). O valor de utilidade de um estado, dada uma política, é o reforço esperado partindo do estado e seguindo a política apresentada na equação 4.3. E paralelamente a essa função valor-estado, existe uma função valor-ação para a política π, que é definida pela equação 4.4.

As funções valores V π e Q π podem ser estimadas por experiências. Por exemplo, se um agente seguir uma política π e mantiver uma média, para cada estado encontrado, dos atuais retornos que tem seguido aquele estado, então a média convergirá ao valor-estado V π . Se médias diferentes forem mantidas para cada ação feita em um estado, então estas médias convergirão similarmente aos valores da ação Q π.

Uma propriedade fundamental das funções de valor usadas durante a Aprendizagem por Reforço e Programação Dinâmica é que elas satisfazem particularidades recursivas:

$$
\begin{aligned}
V^{\pi} &= E_{\pi}\{\sum_{k=0}^{\infty}\gamma^{k} r_{t+k+1} \mid s_t = s\} \\
&= E_{\pi}\{r_{t+1} + \gamma\sum_{k=0}^{\infty}\gamma^{k} r_{t+k+2} \mid s_t = s\} \\
&= \sum_{a\in A(s)} \pi(s,a) \sum_{s'\in S} P^{a}_{s,s'} \left[R^{a}_{s,s'} + \gamma E_{\pi}\left\{\sum_{k=0}^{\infty}\gamma^{k} r_{t+k+2} \mid s_{t+1} = s'\right\}\right] \\
&= \sum_{a\in A(s)} \pi(s,a) \sum_{s'\in S} P^{a}_{s,s'} \left[R^{a}_{s,s'} + \gamma V^{\pi}(s')\right], \ \forall s \in S
\end{aligned}
$$

A equação acima é a equação de Bellman para V π . Expressa um relacionamento entre o valor de um estado e os valores de seus estados sucessores. Calcula todas possíveis médias excessivamente, ponderando cada um por sua probabilidade de ocorrer. Indica que o valor do estado s inicial deve igualar ao valor amortizado do estado seguinte previsto, mais a recompensa

esperada ao longo da caminho. O algoritmo para resolver a avaliação de política pode ser visto abaixo.

Avaliação da Politica Iterativa

Entrar π, a politica a ser avaliada
Inicializar $V(s) = 0$ para todo $s \in S$
Repete
  $\Delta \leftarrow 0$
  Para cada $s \in S$:
    $v \leftarrow V(s)$

    $V(s) \leftarrow \sum_{a \in A(s)} \pi(s,a) \sum_{s' \in S} P^a_{ss'} [R^a_{ss'} + \gamma V(s')]$
    $\Delta \leftarrow max(\Delta, | v - V(s) |)$
  ate $\Delta < \theta$ (um pequeno numero positivo)
  Sair $V \approx V^\pi$

Como demonstração, resolveremos o exemplo do robô reciclador. Para facilitar o entendimento adotamos abreviações nos estados (ex: high = h) e nas ações (ex: search = s), e adotamos os valores abaixo na implementação do algoritmo no Matlab, fazendo uma avaliação iterativa da política.

$$
\begin{array}{llllllllllll}
R^w & = & 1 & , & \alpha & = & 0.8 & , & \pi(l,r) & = & \frac{1}{3} & , \quad \pi(h,w) = \frac{1}{2} \\
R^s & = & 10 & , & \beta & = & 0.2 & , & \pi(l,w) & = & \frac{1}{3} & , \quad \pi(h,s) = \frac{1}{2} \\
R^r & = & 0 & , & \gamma & = & 0.9 & , & \pi(l,s) & = & \frac{1}{3} & , \\
R^{f,r} & = & -3 & , & & & & & & & &
\end{array}
$$

Resolvendo a equação de Bellman, temos:

$$
\begin{aligned}
V^{\pi}(l) &= \sum_{a \in A(l)} \pi(l,a) \sum_{s' \in S} P^{a}_{l,s'} [R^{a}_{l,s'} + \gamma V^{\pi}(s')] \\
&= \pi(l,r) \left\{ P^{r}_{l,h} [R^{r}_{l,h} + \gamma V^{\pi}(h)] + P^{r}_{l,l} [R^{r}_{l,l} + \gamma V^{\pi}(l)] \right\} + \\
&\quad \pi(l,w) \left\{ P^{w}_{l,h} [R^{w}_{l,h} + \gamma V^{\pi}(h)] + P^{w}_{l,l} [R^{w}_{l,l} + \gamma V^{\pi}(l)] \right\} + \\
&\quad \pi(l,s) \left\{ P^{s}_{l,h} [R^{s}_{l,h} + \gamma V^{\pi}(h)] + P^{s}_{l,l} [R^{s}_{l,l} + \gamma V^{\pi}(l)] \right\} \\
&= \frac{1}{3} \{1[0 + 0.9V^{\pi}(h)] + 0[0 + 0.9V^{\pi}(l)]\} + \\
&\quad \frac{1}{3} \{0[0 + 0.9V^{\pi}(h)] + 1[1 + 0.9V^{\pi}(l)]\} + \\
&\quad \frac{1}{3} \{(1-\beta)[-3 + 0.9V^{\pi}(h)] + \beta[10 + 0.9V^{\pi}(l)]\} \\
&= 0.47 + 0.54V^{\pi}(h) + 0.36V^{\pi}(l)
\end{aligned}
$$

$$
\begin{aligned}
V^{\pi}(h) &= \sum_{a \in A(h)} \pi(h,a) \sum_{s' \in S} P^{a}_{h,s'} [R^{a}_{h,s'} + \gamma V^{\pi}(s')] \\
&= \pi(h,w) \left\{ P^{w}_{h,h} [R^{w}_{h,h} + \gamma V^{\pi}(h)] + P^{w}_{h,l} [R^{w}_{h,l} + \gamma V^{\pi}(l)] \right\} + \\
&\quad \pi(h,s) \left\{ P^{s}_{h,h} [R^{s}_{h,h} + \gamma V^{\pi}(h)] + P^{s}_{h,l} [R^{s}_{h,l} + \gamma V^{\pi}(l)] \right\} \\
&= \frac{1}{2} \{1[1 + 0.9V^{\pi}(h)] + 0[0 + 0.9V^{\pi}(l)]\} + \\
&\quad \frac{1}{2} \{\alpha[10 + 0.9V^{\pi}(h)] + (1-\alpha)[10 + 0.9V^{\pi}(l)]\} \\
&= 5.5 + 0.81V^{\pi}(h) + 0.09V^{\pi}(l)
\end{aligned}
$$

Resolvendo as equações lineares acima no Matlab, obtivemos os seguintes resultados: V π (l) = 45.19 e V π (h) = 50.35, que indicam a solução encontrada usando a equação de Bellman através do cálculo iterativo para as seqüências de V π (s).

Política Ótima

A Programação Dinâmica organiza e estrutura a busca de boas políticas a partir das funções de valor. Deste modo, políticas ótimas são obtidas sempre que funções de valor ótimas são obtidas. Usualmente, as funções valor-estado

ótimas são denotadas por V ∗ (s) e valor-ação Q ∗ (s,a) as quais satisfazem as equações de otimização de Bellman, como é expresso nas equações abaixo, respectivamente:

$$
\begin{aligned}
V^*(s) &= \max_{a \in A(s)} Q^{\pi^*}(s,a) \\
&= \max_a E_{\pi^*}\{R_t \mid s_t = s, a_t = a\} \\
&= \max_a E_{\pi^*}\left\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a\right\} \\
&= \max_a E_{\pi^*}\left\{r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s, a_t = a\right\} \\
&= \max_a E\{r_{t+1} + \gamma V^*(s_t + 1) \mid s_t = s, a_t = a\} \\
&= \max_a \sum_{s'} P^a_{ss'}[R^a_{ss'} + \gamma V^*(s')]
\end{aligned}
$$

$$
\begin{aligned}
Q^*(s,a) &= E\left\{r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a\right\} \\
&= \sum_{s'} P^a_{ss'}[R^a_{ss'} + \gamma \max_{a'} Q^*(s', a')]
\end{aligned}
$$

Na Equação a função de valor ótimo V ∗ (s) é encontrada como o máximo das funções de valor esperadas segundo a ação selecionada. E a partir de Q ∗ , pode-se determinar uma política ótima simplesmente como π ∗ (s) = argmaxa∈A(s) Q ∗ (s,a). Para atualizar as funções valor com a finalidade de melhorar a política, utiliza-se a iteração de valores. A função valor Vk+1(s) do estado s para o passo k + 1 de avaliação é encontrada como na equação:

$$
\begin{aligned}
V_{k+1}(s) &= \max_a E\{r_{t+1} + \gamma V_k(s_{t+1}) \mid s_t = s, a_t = a\} \\
&= \max_a \sum_{s'} P^a_{s,s'}[R^a_{s,s'} + \gamma V_k(s')], \forall s \in S.
\end{aligned}
$$

onde o valor atualizadoVk+1(s) é encontrado a partir dos valores armazenados no passo k da seqüência de iterações, aplicando a equação de otimalidade de Bellman (4.6). Esta seqüência de iterações deve alcançar no ponto final a política ótima V ∗ (s).

O método de procura da política ótima da Programação Dinâmica exige a varredura de todos os estados no espaço de estados do modelo PDM, fazendo com que exista um grande custo computacional para modelagens complexas, resultando em uma desvantagem do método. O termo gulosa (greedy) é usado para descrever um procedimento de busca ou de decisão, que seleciona alternativas baseadas somente em considerações locais e/ou imediatas, sem considerar a possibilidade que tal seleção poder encontrar no futuro alternativas melhores. Conseqüentemente, descreve as políticas que as ações baseiam somente em conseqüências à curto prazo.

VALOR DE ITERAÇÃO

Inicializar V de forma arbitraria, por exemplo $V(s) = 0$, para todo $s \in S$

Repete
  $\Delta \leftarrow 0$
  Para cada $s \in S$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \max_{a \in A(s)} \sum_{s' \in S} P^a_{ss'} [R^a_{ss'} + \gamma V(s')]$
    $\Delta \leftarrow max(\Delta, | v - V(s) |)$
  ate $\Delta < \theta$ (um pequeno numero positivo)

Sair uma politica deterministica, π, tal que
  $\pi(s) = arg \max_{a \in A(s)} \sum_{s' \in S} P^a_{ss'} [R^a_{ss'} + \gamma V(s')]$

Continuando com o exemplo do robô reciclador, resolveremos as equações de Otimalidade de Bellman, assim temos para a função valor-estado:

$$
\begin{aligned}
V^*(l) &= \max_{a \in A(s)} \sum_{s' \in S} P^a_{s,s'} [R^a_{s,s'} + \gamma V^*(s')] \\
&= \max \left\{ \begin{array}{c} P^r_{l,h}[R^r_{l,h} + \gamma V^*(h)] + P^r_{l,l}[R^r_{l,l} + \gamma V^*(l)], \\ P^w_{l,h}[R^w_{l,h} + \gamma V^*(h)] + P^w_{l,l}[R^w_{l,l} + \gamma V^*(l)], \\ P^s_{l,h}[R^s_{l,h} + \gamma V^*(h)] + P^s_{l,l}[R^s_{l,l} + \gamma V^*(l)] \end{array} \right\} \\
&= \max \left\{ \begin{array}{c} 0.9V^*(h), \\ 1 + 0.9V^*(l), \\ -0.4 + 0.72V^*(h) + 0.18V^*(l) \end{array} \right\} \\
V^*(h) &= \max_{a \in A(s)} \sum_{s' \in S} P^a_{s,s'} [R^a_{s,s'} + \gamma V^*(s')] \\
&= \max \left\{ \begin{array}{c} P^w_{h,h}[R^w_{h,h} + \gamma V^*(h)] + P^w_{h,l}[R^w_{h,l} + \gamma V^*(l)]\}, \\ P^s_{h,h}[R^s_{h,h} + \gamma V^*(h)] + P^s_{h,l}[R^s_{h,l} + \gamma V^*(l)]\} \end{array} \right\} \\
&= \max \left\{ \begin{array}{c} 1 + 0.9V^*(h), \\ 10 + 0.72V^*(h) + 0.18V^*(l) \end{array} \right\}
\end{aligned}
$$

e, para a função valor-ação:

$$
\begin{aligned}
Q^*(l,r) &= \sum_{s' \in S} P^r_{l,s'}[R^r_{l,s'} + \gamma \max_{a' \in A(s)} Q^*(s',a')] \\
&= P^r_{l,h}[R^r_{l,h} + \gamma \max\{Q^*(h,w), Q^*(h,s)\}] \\
&= 0.9 \max\{Q^*(h,w), Q^*(h,s)\}
\end{aligned}
$$

$$
\begin{aligned}
Q^*(l,w) &= \sum_{s' \in S} P^w_{l,s'}[R^w_{l,s'} + \gamma \max_{a' \in A(s)} Q^*(s',a')] \\
&= P^w_{l,l}[R^w_{l,l} + \gamma \max\{Q^*(l,w), Q^*(l,s), Q^*(l,r)\}] \\
&= 1 + 0.9 \max\{Q^*(l,w), Q^*(l,s), Q^*(l,r)\}
\end{aligned}
$$

$$
\begin{aligned}
Q^*(l,s) &= \sum_{s' \in S} P^s_{l,s'}[R^s_{l,s'} + \gamma \max_{a' \in A(s)} Q^*(s',a')] \\
&= P^s_{l,l}[R^s_{l,l} + \gamma \max\{Q^*(l,w), Q^*(l,s), Q^*(l,r)\}] + P^s_{l,h}[R^s_{l,h} + \gamma \max\{Q^*(h,w), Q^*(h,s)\}] \\
&= -0.4 + 0.18 \max\{Q^*(l,w), Q^*(l,s), Q^*(l,r)\}] + 0.72 \max\{Q^*(h,w), Q^*(h,s)\}]
\end{aligned}
$$

$$
\begin{aligned}
Q^*(h,w) &= \sum_{s' \in S} P^w_{h,s'}[R^w_{s,s'} + \gamma \max_{a' \in A(s)} Q^*(s',a')] \\
&= P^w_{h,h}[R^w_{h,h} + \gamma \max\{Q^*(h,w), Q^*(h,s)\}] \\
&= 1 + 0.9 \max\{Q^*(h,w), Q^*(h,s)\}
\end{aligned}
$$

$$
\begin{aligned}
Q^*(h,s) &= \sum_{s' \in S} P^s_{h,s'}[R^s_{s,s'} + \gamma \max_{a' \in A(s)} Q^*(s',a')] \\
&= P^s_{h,l}[R^s_{h,l} + \gamma \max\{Q^*(l,w), Q^*(l,s), Q^*(l,r)\}] + P^s_{h,h}[R^s_{h,h} + \gamma \max\{Q^*(h,w), Q^*(h,s)\}] \\
&= 10 + 0.18 \max\{Q^*(l,w), Q^*(l,s), Q^*(l,r)\}] + 0.72 \max\{Q^*(h,w), Q^*(h,s)\}]
\end{aligned}
$$

Onde obtivemos no Matlab os seguintes resultados: V $*$ (l) = 91.98, V $*$ (h) = 88.41, Q $*$ (l,r) = 79.57, Q $*$ (l,w) = 74.78, Q $*$ (l,s) = 81.98, Q $*$ (h,w) = 80.57 e Q $*$ (h,s) = 88.41. Observe que os valores V $*$ (l) e V $*$ (h) são bem superiores aos obtidos com a política equiprovável π, V π (l) = 45.19 e V π (h) = 50.35, conforme resolvidos acima. A política π $*$ obtida a partir de Q $*$ é Q $*$ (h,s) = 88.41.

Monte Carlo (MC)

O método de Monte Carlo [37] não precisa da modelagem do ambiente e se apresenta de forma simples em termos conceituais, baseia-se no cálculo da média de retornos obtidos em seqüências. Para assegurar-se que exista um valor de retorno bem definido, o método de Monte Carlo é utilizado apenas para tarefas episódicas, isto é, a experiência é dividida em episódios que de algum

modo alcançam o estado final sem importar as ações que foram selecionadas (exemplo: jogo de xadrez). Desta forma, somente depois da conclusão de um episódio o valor de retorno é obtido e as políticas são atualizadas. Entretanto, não são viáveis quando a solução do problema é possível apenas de forma incremental, porque para se atualizar, o método de Monte Carlo exige que seja alcançado o estado final no processo e com isso o mesmo pode se apresentar lento.

Uma vantagem do método de Monte Carlo é que, diferente do método de Programação Dinâmica, não necessita de informação completa do ambiente, apenas necessita das amostras da experiência como seqüências de dados, ações e reforços a partir de uma interação real ou simulada com o ambiente. O aprendizado a partir de experiência real é notável, uma vez que não exige o conhecimento a priori das dinâmicas do ambiente, e ainda, pode levar a um comportamento ótimo. Embora seja requerida uma modelagem, esta deve apenas gerar transições de estados, sem precisar todo o conjunto de distribuições de probabilidade para todas as possíveis transições, como é exigido pela Programação Dinâmica.

Avaliação da Política

Supondo-se que o método de Monte Carlo é considerado para obter a função valor sob uma dada política, que é representada pelo retorno esperado, isto é, a acumulação amortizada dos futuros reforços desde o estado s até o estado desejado. Uma forma de se aproximar do valor de retorno esperado a partir da experiência é calcular a média dos retornos observados após visitar esse estado. Na medida em que mais retornos são observados, a média deve se aproximar do valor real esperado, sendo esta conseqüência o princípio básico do método de Monte Carlo.

Seja $V\pi(s)$ a função valor-estado sob a política π. Dados um conjunto de episódios obtidos sob a mesma política passando pelo estado s (cada ocorrência de s em um episódio é chamada de visita a s), existem duas variantes do método de Monte Carlo: A primeira, chamada de Every-Visit MC Method, estima a função de valor como a média dos retornos após todas as visitas ao estado s, enquanto

a segunda, chamada de First-Visit MC Method, estima a função de valor como a média dos retornos após a primeira visita ao estado s.

De qualquer forma, se o número de visitas for infinito, ambas as variantes do método de Monte Carlo, convergem ao valor V π (s). Podemos ver através do algoritmo abaixo.

PRIMEIRA VISITA AO METODO MC PARA ESTIMAR V Π

```
Inicializar:
  π ← politica a ser avaliada
  V ← uma função valor-estado arbitraria
  Retornos(s) ← uma lista vazia, para todo s ∈ S

Repetir sempre:
  (a) Gerar um episodio usando π

(b) Para cada estado s que aparece no episodio:
  R ← quantidade de retorno apos a primeira ocorrência de s
  Adiciona R a lista Retornos(s)
  V(s) ← media de (Retornos(s))
```

Resolvendo o algoritmo para o exemplo do robô reciclador, onde definiu-se como episódio uma seqüência de 2000 passos de iterações, obtivemos um comportamento que pode ser observado na Figura 4.4, com resultados bem próximos ao obtido na PD, V π (l) = 45.43 e V π (h) = 50.32.

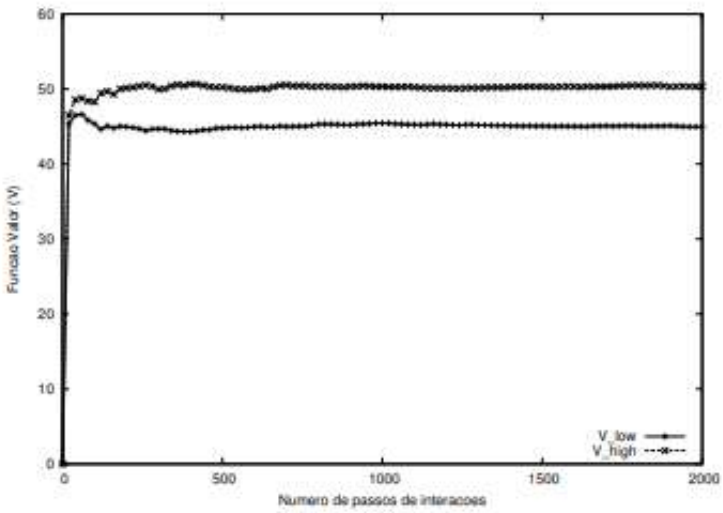


Figura 4.4: Comportamento do algoritmo MC para a avaliação de política

Política Ótima

A fim de melhorar a política é necessário fazer com que esta seja mais gulosa para a função valorestado V π (s) atual. Neste caso é conveniente assumir como valor de retorno a função valor-ação Q π (s,a). Assim, uma política gulosa para uma função valor-ação Q(s,a) é aquela que para um estado s toma a ação que maximiza o valor Q como na equação abaixo.

$$\pi(s) = arg \max_{a \in A(s)} Q(s,a)$$

Desta forma, uma melhora na política pode ser obtida fazendo a política πk+1 ser gulosa em respeito à função valor-ação Q πk , logo após a avaliação da função valor-ação Q πk de πk, podemos gerar uma seqüência de avaliação da função e melhora da política, abaixo:

$$\pi_0 \xrightarrow{A} Q^{\pi_0} \xrightarrow{M} \pi_1 \xrightarrow{A} Q^{\pi_1} \ldots \xrightarrow{A} Q^{\pi_k} \xrightarrow{M} \pi_{k+1} \ldots \xrightarrow{M} \pi^* \xrightarrow{A} Q^*$$

Onde A indica avaliação de política e M indica processo guloso de melhora de política. Segundo este processo se o número de episódios é muito grande, a função valor se aproximará à função valor-ação ótima Q ∗ .

MC COM EXPLORAÇÃO NO INICIO

```
Inicializa, para todo s ∈ S, a ∈ A(s):
  Q(s,a) ← de forma arbitraria
  π(s) ← de forma arbitraria
  Retorno(s,a) ← lista vazia para s ∈ S

Repetir infinitamente:
  (a) Gerar um episodio usando π explorando novos estados
  (b) Para cada par (s,a) gerado no episodio:
    R ← retorno apos a primeira ocorrência de (s,a)
    Adiciona R a lista Retorno(s, a)
    Q(s,a) ← media de (Retorno(s,a))
  (c) Para cada s do episodio:
    π(s) ← arg max_a Q(s,a)
```

Diferença Temporal (DT)

Os métodos de Diferenças Temporais não exigem um modelo exato do sistema e permitem ser incrementais, da mesma forma que os métodos de Monte Carlo. Os métodos de Diferenças Temporais são uma combinação de características dos métodos de Monte Carlo com as idéias da Programação Dinâmica, que buscam estimar valores de utilidade para cada estado no ambiente [41]. Em outras palavras, quanto mais próximo da convergência do método, mais o agente tem certeza de qual ação tomar em cada estado.

O aprendizado é feito diretamente a partir da experiência, sem a necessidade de uma modelagem completa do ambiente, como característico do método de Monte Carlo, mas leva vantagem em cima deste por atualizar as estimativas da função valor a partir de outras estimativas já aprendidas em estados sucessivos (bootstrap), sem a necessidade de alcançar o estado final de um episódio antes da atualização. Neste caso a avaliação de uma política é abordada como um problema de predição, isto é, estimar a função valor V π sob a política π.

Avaliação da Política - Predição DT

Tanto DT como MC utilizam a experiência para resolver o problema da predição. Dada certa experiência sob a política π, se é visitado um estado intermediário st , ambos os métodos atualizam suas estimativas V π (st) baseando-se no acontecido depois de visitado o estado. Sendo que o método de Monte Carlo espera até que o retorno total seja conhecido e usa esse retorno como objetivo para a atualização de V π (st), como aparece na equação abaixo.

$$V^{\pi}(s_t) \leftarrow V^{\pi}(s_t) + \alpha[R_t - V^{\pi}(s_t)]$$

onde Rt representa o retorno atual no instante t, e o símbolo α é uma constante de atualização (taxa de aprendizagem), α ∈ [0,1].

Os métodos de Diferenças Temporais não necessitam alcançar o estado final de um episódio, e sim o estado seguinte no instante t + 1. Em DT são utilizados, o valor de reforço imediato rt+1 e a função de valor estimada V π

(st+1) para o próximo estado ao invés do valor real de retorno Rt como no método de Monte Carlo, executando a atualização imediatamente após cada passo. Com estas condições, nos métodos de Diferenças Temporais a Equação 4.10 converte-se na Equação abaixo.

$$V^{\pi}s_t \leftarrow V^{\pi}s_t + \alpha[r_{t+1} + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)]$$

onde o objetivo para atualização é o valor rt+1 +γV π (st+1)−V π (st) que precisamente define a diferença no tempo t e t +1 , característica esta que neste método recebe o nome de Diferenças Temporais. Como a atualização é feita a partir do estado seguinte, os métodos DT são conhecidos como métodos single-step.

PREDIÇÃO DT PARA ESTIMAR V Π

```
Inicializar V(s) de forma arbitraria, e π (politica a ser avaliada)
Repete (para cada episodio):
  Inicializar s
  Repete (para cada passo do episodio):
    a ← ação dada por π para s
    Tomar a ação a, observar retorno r e proximo estado s'
    V(s) ← V(s) + α[r + γV(s') − V(s)]
    s ← s'
  ate s ser o estado final
```

Para o exemplo do robô reciclador, implementamos o algoritmo acima, definindo-se como episódio uma seqüência de 2000 passos de iterações, e o comportamento obtido pode ser acompanhado na Figura 4.5, onde obtivemos os seguintes resultados finais: V(l) = 45.86 e V(h) = 50.55, que são bem próximos aos valores obtidos na Programação Dinâmica e no método de Monte Carlo.

Vantagens dos Métodos de Predição DT

A vantagem mais notável do método DT é a relacionada com o método de Programação Dinâmica, onde esta não necessita da modelagem PDM do ambiente, de seus reforços e das distribuições de probabilidade das transições dos seus estados.

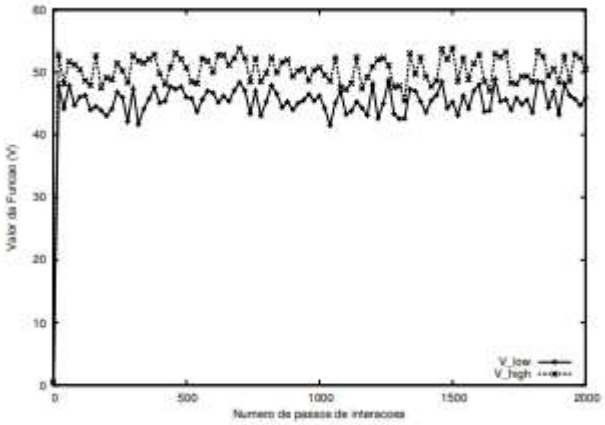


Figura 4.5: Comportamento do algoritmo Predição DT para a avaliação de política

A vantagem seguinte diz respeito ao método de Monte Carlo, visto que DT pode ser implementado de forma totalmente incremental para aplicações On-Line; o método de Monte Carlo deve aguardar até o final de um episódio para obter o retorno verdadeiro, enquanto DT só necessita aguardar até o estado seguinte. Em aplicações em que os ambientes são definidos como sendo contínuo, o conceito de episódio não é aplicável com facilidade.

Embora as atualizações das funções valor não sejam feitas a partir de reforços reais, mas de valores supostos, é garantida a convergência até a resposta correta. Tanto em DT como em MC a convergência às predições corretas tem forma assintótica. Dentre os dois métodos, algum deles devem convergir mais rápido; a resposta ainda não é dada formalmente, uma vez que até este momento não existe uma demonstração matemática de qual dos métodos é o mais rápido. Mesmo assim, é mostrado experimentalmente que os métodos DT são mais rápidos para tarefas estocásticas [44].

Q-learning

Um dos maiores avanços na área de AR foi o desenvolvimento de um algoritmo baseado em Diferenças Temporais que dispensa a política, (off-policy methods) conhecido como Q-learning. A versão mais simples, One-step Q-learning [48], é definida pela seguinte expressão:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha[r_{t+1} + \gamma \ max_a Q(s_{t+1},a) - Q(s_t,a_t)]$$

onde a função de valor-ação Q(st ,at) é atualizada a partir do seu valor atual, o reforço imediato rt+1, e a diferença entre a máxima função valor no estado seguinte (encontrando e selecionando a ação do estado seguinte que a maximize), menos o valor da função valor-ação no tempo atual. O fato de selecionar a ação que maximize a função valor no estado seguinte permite achar de uma forma simples a função valor-ação estimada.

Uma característica do Q-learning é que a função valor-ação Q aprendida, aproxima-se diretamente da função valor-ação ótima Q $*$ sem depender da política que está sendo utilizada. Este fato simplifica bastante a análise do algoritmo e permite fazer testes iniciais da convergência. A política ainda mantém um efeito ao determinar quais pares estado-ação devem ser visitados e atualizados, porém, para que a convergência seja garantida, é necessário que todos os pares estado-ação sejam visitados continuamente e atualizados, por isso Q-learning é um método off-policy [44].

ALGORITMO Q-LEARNING

```
Inicializar Q(s,a) de forma arbitraria
Repete (para cada episodio):
  Inicializar s
  Repete (para cada passo do episodio):
    Escolher a para s usando politica obtida dado Q (e.g, ε-gulosa)
    Tomar a ação a, observar o proximo estado s', e o retorno r
    Q(s,a) ← Q(s,a) + α[r + γmax_{a'∈A(s')} Q(s',a') − Q(s,a)]
    s ← s'
  ate s ser o estado final
```

A política ε-gulosa é definida no algoritmo pela escolha da ação que possui o maior valor esperado, com a probabilidade definida por (1 − ε), e de ação aleatória, com probabilidade ε. Este processo permite que o algoritmo explore o espaço de estados e esta é uma das condições necessárias para garantir que algoritmos RL encontrem a ação ótima. Para o exemplo do robô reciclador, implementamos o algoritmo Q-learning onde cada episódio foi definido como uma seqüência de 5000 passos de iterações, e o comportamento

das 1500 primeiras iterações pode ser observado nas Figuras 4.6 e 4.7, onde obtivemos os seguintes resultados finais: Q(l,w) = 74.37, Q(l,s) = 81.46, Q(l,r) = 79.70, Q(h,w) = 79.96 e Q(h,s) = 88.24 que são bem próximos aos valores obtidos na Programação Dinâmica. E com isso constatar que a política ótima $\pi *$ obtida a partir de $Q *$ é Q(h,s) = 88.24.

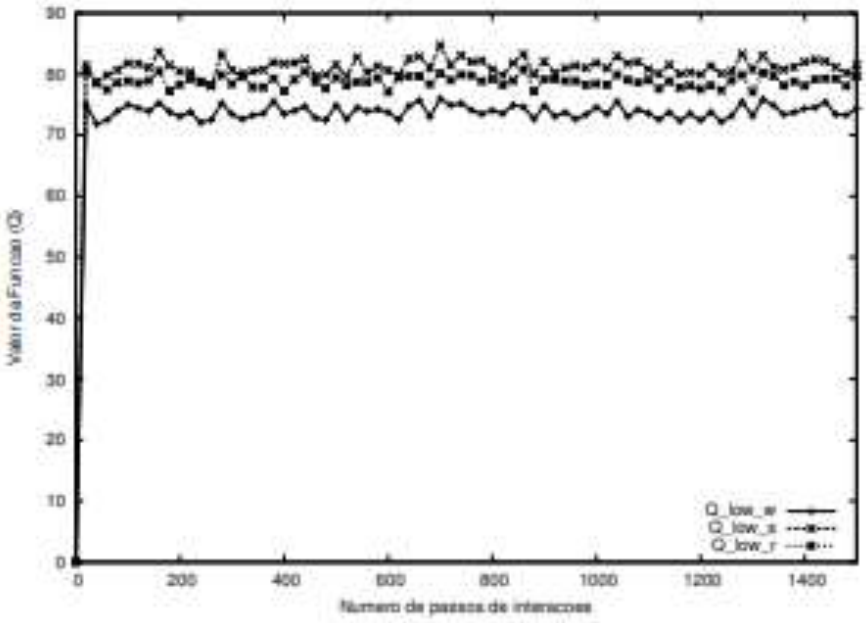


Figura 4.6: Comportamento do algoritmo Q-learning para os valores Q(l)

Q-learning foi o primeiro método AR a possuir fortes provas de convergência. É uma técnica muito simples que calcula diretamente as ações sem avaliações intermediárias e sem uso de modelo.

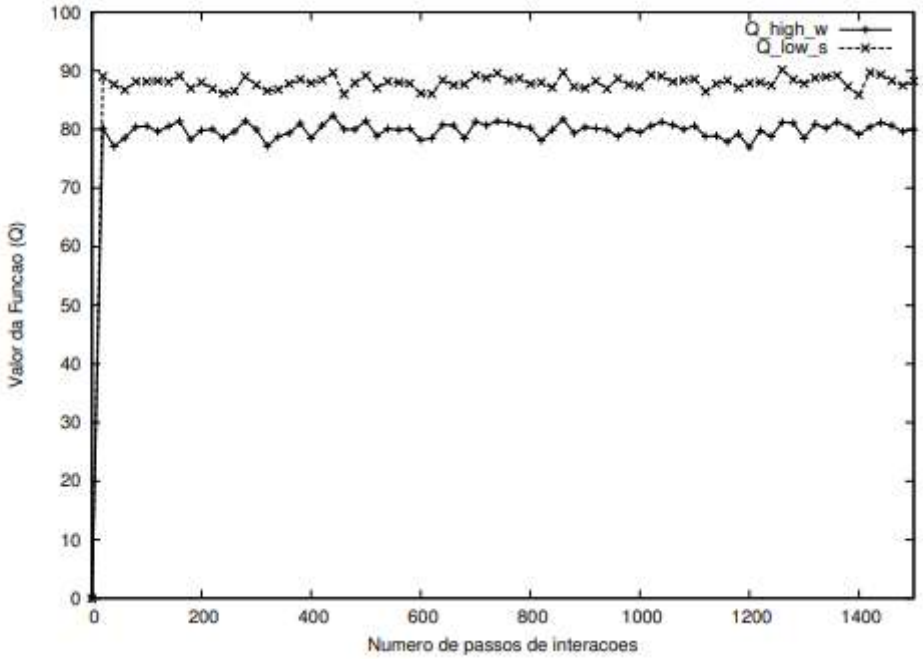


Figura 4.7: Comportamento do algoritmo Q-learning para os valores Q(h)

Dados os valores Q, existe uma política definida pela execução da ação a, quando o agente esta em um estado s, que maximiza o valor Q(s,a). Em [48] é mostrado que se cada par estado-ação for visitado um número suficientemente grande de vezes e α decrescer apropriadamente, asfunções valoração Q irão convergir com probabilidade um para Q ∗ e, conseqüentemente, a política irá convergir para uma política ótima. A convergência do algoritmo Q-learning não depende do método de exploração usado. Um agente pode explorar suas ações a qualquer momento, não existem requisitos para a execução de ações estimadas como as melhores. No entanto, para melhorar o desempenho do sistema é necessária, durante o aprendizado, a busca das ações que maximizam o retorno [21].

Resumidamente, pode-se enumerar os mais importantes aspectos do algoritmo Q-learning:

• O objetivo do uso do algoritmo Q-learning é achar uma regra de controle que maximize cada ciclo de controle;

• O uso do reforço imediato é indicado sempre que possível e necessário, desde que ele contenha informação suficiente que ajude o algoritmo a achar a melhor solução;

• Q-learning é adotado quando o número de estados e ações a serem selecionados é finito e pequeno.

Considerações Finais

Neste capítulo foi visto que os problemas em Aprendizagem por Reforço são caracterizados por um agente que deve aprender comportamentos através de interações de tentativa e erro em um ambiente dinâmico, ou seja, se uma ação desse é seguida de estados satisfatórios, ou por uma melhoria no estado, então a tendência para produzir esta ação é reforçada. Foi visto também que AR não é definido como um conjunto de algoritmo de aprendizagem, mas como uma classe de problemas de aprendizagem e que todo algoritmo que resolver bem esse problema será considerado um algoritmo de AR [18].

Apresentou-se a seus fundamentos matemáticos, através da Propriedade de Markov e do Processo de Decisão Markoviano que é quando uma tarefa de AR satisfaz as Propriedades de Markov. Foi visto que a AR dispõe de vários métodos de aprendizagem, apresentou-se as características desses métodos, e através de um exemplo estudou-se seus métodos de resolução, avaliando e comparando suas eficiências entre si, com o objetivo de salientar similaridades e diferenças entre estas teorias.

Foi apresentado o algoritmo Q-learning, algoritmo adotado no estudo dessa dissertação, por ter uma facilidade de aproximação da função ótima sem depender da política que está sendo utilizada, simplificando a analise e permitindo testes iniciais de convergência.

ESTUDO DE CASO I

Utilização de Aprendizagem por Reforço para Modelagem Autônoma do Aprendiz em um Tutor Inteligente

INTRODUÇÃO

Um Sistema Tutor Inteligente (STI) é uma evolução de sistemas CAI (Computer-Assisted Instruction), aperfeiçoado com as técnicas de Inteligência Artificial (IA). Os STI possibilitam ao estudante a capacidade de aprender com um tutor, que serve como guia no processo. Ele deve se adaptar ao aprendiz, e não o contrário, como acontece no método tradicional. Com isso, é necessário um modelamento do aprendiz, para que o STI possa saber o que ensinar, a quem ensinar e como ensinar. De acordo com [Marietto, 2000] o STI deve ser capaz de mudar o nível de entendimento para responder às entradas do aprendiz, em vários níveis, podendo mudar as estratégias pedagógicas, de forma individualizada (de acordo com o ritmo e as características de cada aprendiz).

Em um STI o ensino é construído sobre uma base de conhecimento (domínio) criada por um especialista. Através da interação com o aprendiz, um STI modifica suas bases de conhecimentos, percebe suas intervenções, e possui a capacidade de aprender e adaptar-se às estratégias de ensino, de acordo com o desenrolar do diálogo com o aprendiz [Leite, 1999]. [Burns, Parllet & Redfield, 1991] salientam que as pesquisas de STIs, principalmente as de ensinoaprendizagem, devem enfocar as estratégias de ensino em particular, dando ênfase aos estados cognitivos de cada aprendiz.

Para ser inteligente, um tutor deve ser flexível, isto é, ter capacidade para aprender com o meio ambiente e atualizar seu conhecimento [Viccari, 1990]. Para aumentar e facilitar a autonomia de um tutor, este artigo propõe um processo de definição de perfil e o uso de um algoritmo de Aprendizado por Reforço (algoritmo Q-learning), através da introdução de um módulo de diagnóstico em uma arquitetura de STI, com o propósito de incorporar ao tutor a capacidade de modelar autonomamente o aprendiz.

O trabalho está organizado como segue. Na Seção 2 apresentam-se os conceitos de Aprendizado por Reforço e o algoritmo Q-Learning. A Seção 3 contém a estrutura do sistema utilizando AR e todos os pontos relevantes para o funcionamento do sistema. A Seção 4 contém experimentos e resultados. Finalmente, a Seção 5 apresenta as vantagens e desvantagens do método proposto, e sugere trabalhos futuros.

TÉCNICA DE APRENDIZADO POR REFORÇO

Afirma [Hendley,1992] que o uso de técnicas de Inteligência Artificial na construção de software educacionais torna-se cada vez mais importante, uma vez que seu estudo fará com que os mesmos tenham mais qualidade. A arquitetura descrita neste trabalho utiliza uma técnica de Inteligência Artificial que é o Aprendizado por Reforço (AR), para modelar o aprendiz de forma dinâmica e autônoma. AR permite ao agente adquirir uma capacidade de conhecimento do ambiente que não estava disponível em tempo de projeto [Sutton & Barto, 1998]. AR é baseada na existência de um crítico externo ao ambiente, que avalia a ação tomada, mas sem indicar explicitamente a ação correta. Formalmente, AR utiliza uma estrutura composta de estados, ações e recompensas conforme mostra a Figura1.

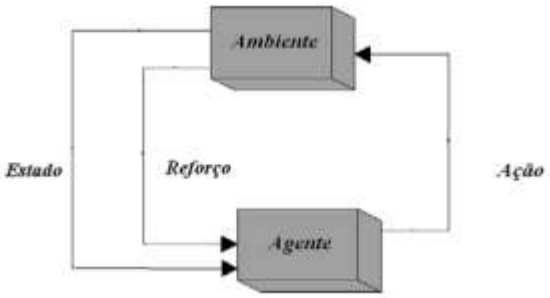


Figura 1. Um agente AR interagindo com seu ambiente.

O agente atua em um ambiente descrito por um conjunto de possíveis estados e pode executar, para cada estado, uma ação dentro de um conjunto de ações possíveis, recebendo um valor de reforços a cada vez que executa uma ação. Este reforço indica o valor imediato da transição estado-ação-novo estado. Ao longo do tempo, este processo produz uma seqüência de pares estado-ação,

e seus respectivos valores de reforços. O objetivo do agente é aprender uma política que maximize uma soma esperada destes reforços a longo prazo.

Uma consideração importante a ser feita quando se lida com AR é conflito exploração X explotação. O agente deve lidar com o problema de haver um compromisso entre escolher a exploração de estados e ações desconhecidos, de modo a coletar nova informação, ou a explotação de estados e ações que já foram aprendidos e que irão gerar altas recompensas, de modo a maximizar seus reforços acumulados [Bellman, 1959]. Sendo assim, por um lado o agente deve aprender quais ações maximizam os valores das recompensas obtidas no tempo, e, por outro, deve agir de forma a atingir esta maximização, explorando ações ainda não executadas ou regiões pouco visitadas do espaço de estados. É importante, portanto, estabelecer uma política mista de explotação e exploração (política ε-greedy), que inclua a escolha intencional (com probabilidade ε) de se executar uma ação que não é considerada a melhor no estado atual, visando a aquisição de conhecimentos a respeito de estados ainda desconhecido ou pouco visitados. A escolha desta ação ocorre de forma aleatória. Em uma política de explotação pura (greedy) escolhem-se as ações que se julguem (talvez erroneamente, caso o algoritmo de AR ainda esteja longe da convergência) serem as melhores para resolver o problema.

No contexto deste trabalho, a técnica de AR terá como função modificar parâmetros e armazená-los em “esquemas de planos”, que definem a forma de apresentar o material instrucional ao aprendiz. O grande desafio, neste caso, é escolher a melhor ação que - baseada no estado do aprendiz - possa mudar a estratégia de ensino, para obter resultados significativos, fornecedores de parâmetros indicativos de quão boa ou ruim está sendo uma determinada estratégia. O algoritmo utilizado (Q-learning) é baseado em estimativas de utilidades de pares estado-ação, e com estas estimativas podem-se gerar alternativas de novas estratégias pedagógicas. Neste caso, a aprendizagem das estratégias mais adequadas, que indicam a ação a escolher para cada estado do aprendiz, dar-se-á por reforço, através de uma estrutura de parâmetros adaptativos sobre os quais o algoritmo opera

Formalmente, AR procura aproximar uma função que define a utilidade relativa dos pares estadoação. Esta função de utilidade fornece indicações estimativas, mapeando os pares estado-ação em uma medida baseada na soma dos reforços esperados a longo prazo. Para cada par estado-ação (s, a), é definido o reforço r(s, a), indicando uma conseqüência imediata da execução da ação a no estado s. O problema em AR é achar uma política ótima de ações (μ*), ou seja, um conjunto de ações que maximizem, para cada estado s, os valores de utilidade Q(s, a). Com base nesses valores, o algoritmo de AR estima a ação de maior valor de utilidade – a ação que deverá ser executada pelo agente tutor.

Q-Learning

O algoritmo Q-Learning [Watkins,1989], pode ser usado para definir a escolha da melhor ação em AR. No Q-Learning, a escolha de uma ação é baseada em uma função de utilidade que mapeia estados e ações a um valor numérico. Neste artigo, o valor de utilidade Q(s, a) de um par (estado, ação), é calculado a partir de reforços medidos pela qualidade do estado cognitivo do aprendiz (modulo de diagnóstico). Cada valor Q(s, a) representa a soma de reforços esperada ao se executar a ação a no estado s, seguindo-se uma política ótima a partir de então. Portanto, uma vez que os valores Q(s, a) estejam bem estimados, a melhor ação a ser executada no estado s pode ser obtida simplesmente como arg maxa Qt(st+1,a). O principal objetivo do algoritmo Q-Learning é estimar autonomamente, em cada estado s em que o aprendiz encontra-se, o valor Q(s, a) para cada possível ação a, e a partir daí permitir a obtenção da melhor ação (ou seja, a ação com maior valor de utilidade).

O Q-Learning normalmente usa uma tabela (Figura 2) para armazenar os valores de utilidade Q(s, a) estimados para os pares (estado, ação).

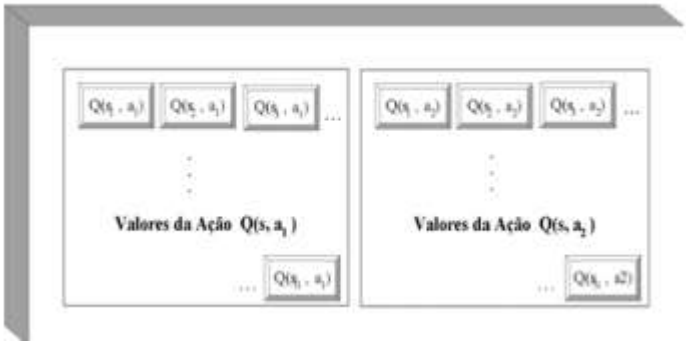


Figura 2. Tabela dos pares (estado - ação).

O algoritmo Q-Learning produz uma atualização dos valores Q da seguinte maneira:

Inicialize **Q(s, a)**.

Para cada instante **t** repita:

1- Observe estado s_t e escolha uma ação a_t;

2- Observe o estado s_{t+1} e atualize $Q_t(s_t, a_t)$ de acordo com

$$Q_{t+1}(s_t,a_t) = Q_t(s_t,a_t) + \alpha_t[r(s_t) + \gamma \max_a Q_t(s_{t+1},a) - Q_t(s_t,a_t)];$$

até **t** igual a limite de passos.

Onde:

• Qt+1(st,at) é o valor (qualidade) da ação at no estado st.

• r(st) é o reforço imediato recebido no estado st.

• α é a taxa de aprendizado (normalmente definida entre 0 e 1).

• γ é a taxa de desconto temporal.

• t É uma seqüência discreta de passos no tempo, ou seja , t=0,1,2,3,....

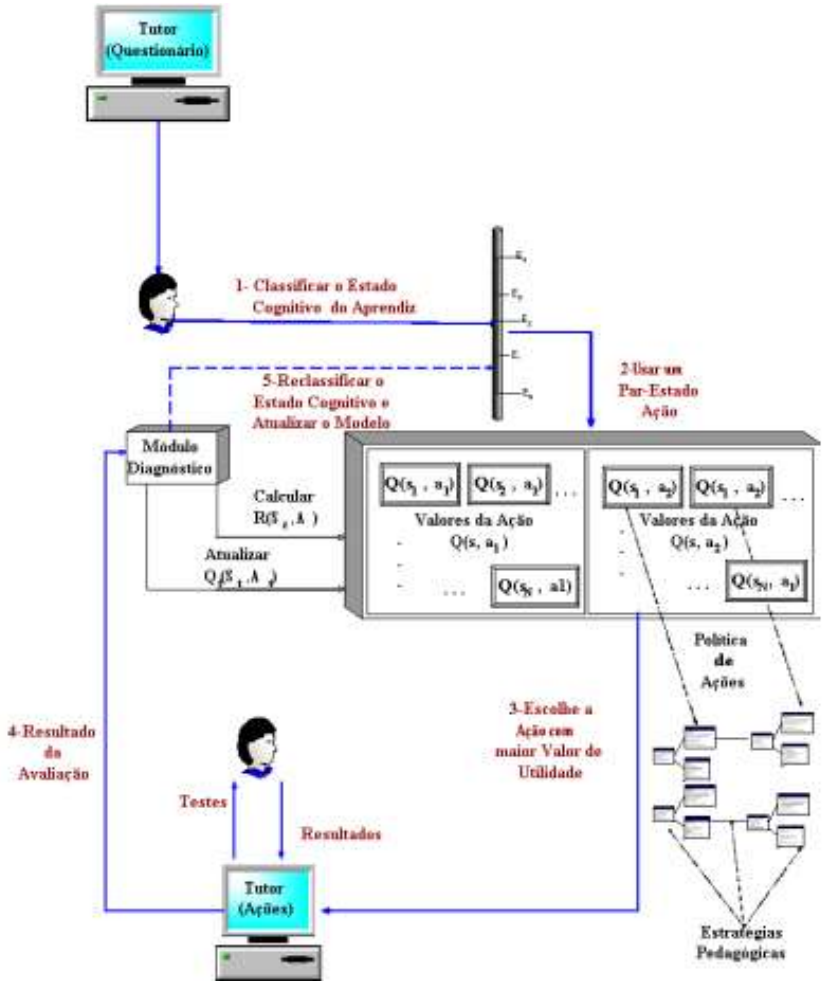
• maxa Qt(st+1,a) é o valor Q correspondente à ação com maior valor de utilidade no estado futuro.

Quanto mais próximo de 1 for o valor de γ, maior importância é dada aos reforços mais distantes no tempo. No início, o algoritmo vai estar muito longe da utilidade ótima associada ao estado s, mas com o passar do tempo as estimativas de Q melhoram. De fato, os valores Q atualizados pelo algoritmo Q-

Learning convergem para os corretos, desde que os pares (s,a) tenham sido visitados um número infinito de vezes [Watkins and Dayan, 1992]. Na prática, convergência para políticas de ação de boa qualidade é obtida apenas com exploração adequada do espaço de estados, durante um número razoável de iterações. A política de ações durante a execução do algoritmo nas fases iniciais de treinamento deve portanto garantir a exploração do espaço de estados (política ε-greedy).

ESTRUTURA DO SISTEMA

A Figura 3 ilustra a estrutura de funcionamento da Técnica de Aprendizado por Reforço para um STI. Todos os itens que fazem parte do sistema serão detalhados a seguir.



Definição do Conjunto dos Estados do Estudante (Perfil)

A complexidade de estabelecer um perfil está na modelagem do aprendiz. O perfil vai indicar o estado cognitivo do mesmo a partir do comportamento observável e, com isso, poder indicar uma estratégia de ensino a ser utilizada. Decisões Pedagógicas razoáveis são fatores de desafio para os tutores, já que as pessoas nem sempre conseguem representar seus próprios processos mentais e, algumas vezes, ficam confusas. De acordo com [Giraffa, 1999] um modelo realista do aluno implica em uma atualização dinâmica enquanto o sistema avalia o desempenho do estudante.

Este modelamento inicialmente dar-se-á através de questionários, onde é conhecido o perfil do aprendiz. Logo em seguida o sistema irá gerar um ciclo que classificará o aprendiz dentro das faixas distintas de conhecimentos (Figura 4), só que agora de forma dinâmica, usando as informações do módulo diagnóstico

Como complementação, para construir um perfil do aprendiz, deve-se manter um log de respostas dadas por este anteriormente, para que ao retornar, seja desnecessária a repetição de tais questionários e possibilite a continuidade do processo.

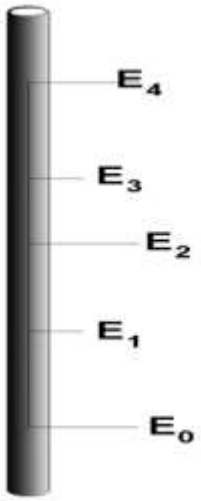


Figura 4. Classificação do Estado Cognitivo do Aprendiz.

A proposta de criação do módulo diagnóstico é de suma importância, pois ela definirá, através da qualidade de sua observação, como se dará o processo de aprendizado. Este módulo diagnóstico produzirá - de acordo com a política

de ações estabelecida – uma indicação de quão bem ou mal está sendo o desempenho do aprendiz em relação às ações produzidas pelo tutor. No contexto de AR, ele enviará reforços indicando o desempenho de modo intermitente, produzindo para cada par (st,a), um valor r(st ,a), correspondendo à reforços positivos quando se produzem resultados favoráveis e reforços negativos para resultados desfavoráveis. Os valores de utilidade Qt(st,at) são atualizados na tabela de valores Q (Figura 2), para que com base nesses valores o tutor possa reclassificar o aprendiz. O sistema classifica o estado cognitivo do Aprendiz através do resultado do questionário, usando uma escala de classificação (Figura 4) e obtendo seu perfil

Com o perfil definido, o sistema usa uma tabela de pares estado-ação. Nesta, as ações representam uma estratégia pedagógica adotada pelo tutor para guiar o aprendiz. O sistema escolhe a ação com o maior valor de utilidade baseado nos resultados do algoritmo QLearning. Estes resultados são encaminhados para o Módulo Diagnóstico, que calcula reforços positivos ou negativos para o par estado-ação (st,a) e atualiza na tabela o valor de utilidade Qt(st,at). Depois de atualizado o valor de utilidade na tabela, o sistema volta a reclassificar o aprendiz na escala.

EXPERIMENTOS

Os experimentos foram realizados sobre um protótipo de tutor, desenvolvido em linguagem C. As simulações foram realizadas no ambiente com uma matriz 5x10 com elementos representados formalmente como: • Um conjunto de estados S={E0, E1, E2, E3, E4}, cada um representando um possível estado cognitivo do aprendiz, em face da interação com o tutor. Estima-se que esse aprendiz tenha um grau cognitivo baseado nesses estados, onde:

E0=> [0,2], E1=>]2,4], E2=>]4,6], E3=>]6,8], E4=>]8,10].

Os valores de 0 a 10, divididos nos cinco conjuntos em cada estado, representam uma métrica usada para diferenciar os estados, seguindo para isso uma escala crescente do grau evolutivo (Figura 4) do aprendiz na interação com o tutor.

• Um conjunto de Ações A={A0, A1, A2, A3, A4, A5, A6, A7, A8, A9} que podem ser escolhidas pelo tutor. Cada ação corresponde à aplicação de provas, exercícios, questionários, perguntas, trabalhos, testes, etc, ou combinações destes e outros dispositivos de avaliação, usados pelo tutor, seguindo as estratégias pedagógicas estabelecidas.

• Um conjunto de Reforços Instantâneos associados a cada estado visitado, ou seja,

E0=>R=1-Ruim;

E1=>R=3-Regular;

E2=>R=5-Bom;

E3=>R=7-Muito Bom;

E4=>R=10-Excelente;

Foi definida uma metodologia de teste na qual foram criados três modelos não determinísticos: M1(Ruim), M2(Bom) e M3(Excelente). Os modelos foram submetidos às simulações com 500, 1000 e 2500 passos, e de cada conjunto de simulação foi calculada uma média sobre vinte realizações, para obtenção dos dados finais. Note que estes modelos não são conhecidos a priori pelo tutor, que deverá estimar por AR uma política de ações em função dos reforços recebidos.

Resultados Obtidos

Os Resultados da Figura 5 mostram as médias dos reforços obtidos durante a simulação com 1000 passos da política pedagógica, para cada um dos modelos M1, M2 e M3 (Não Deterministico) . Q-Learning consegue convergir para a melhor política de ação de cada modelo simulado, usando $\alpha=0.9$ e $\gamma=0.9$.

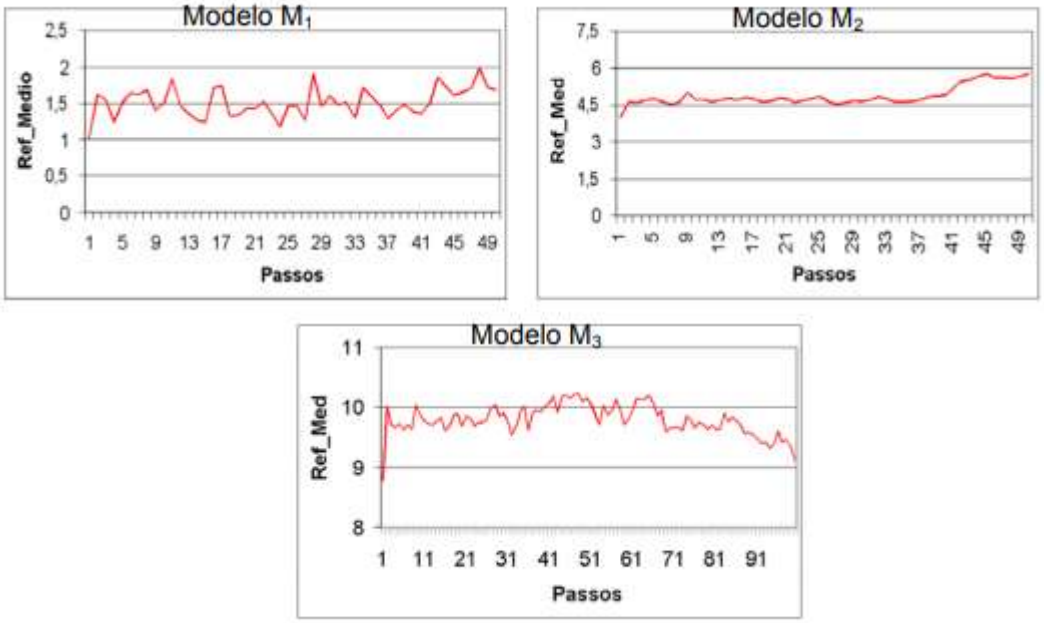


Figura 5. Gráficos dos Reforços Médios dos Modelos M1, M2 e M3.

A Figura 6 demonstra o mesmo conjunto de simulação usado na Figura 5, porém o gráfico mostra os resultados das médias dos valores de utilidade Q(s,a).Com base nesses valores, o algoritmo consegue definir (através da maximização sobre Q(s,a)) a ação de maior valor de utilidade no estado s. Ao longo do tempo ocorreu convergência para uma política ótima de ações (μ) em cada modelo simulado.

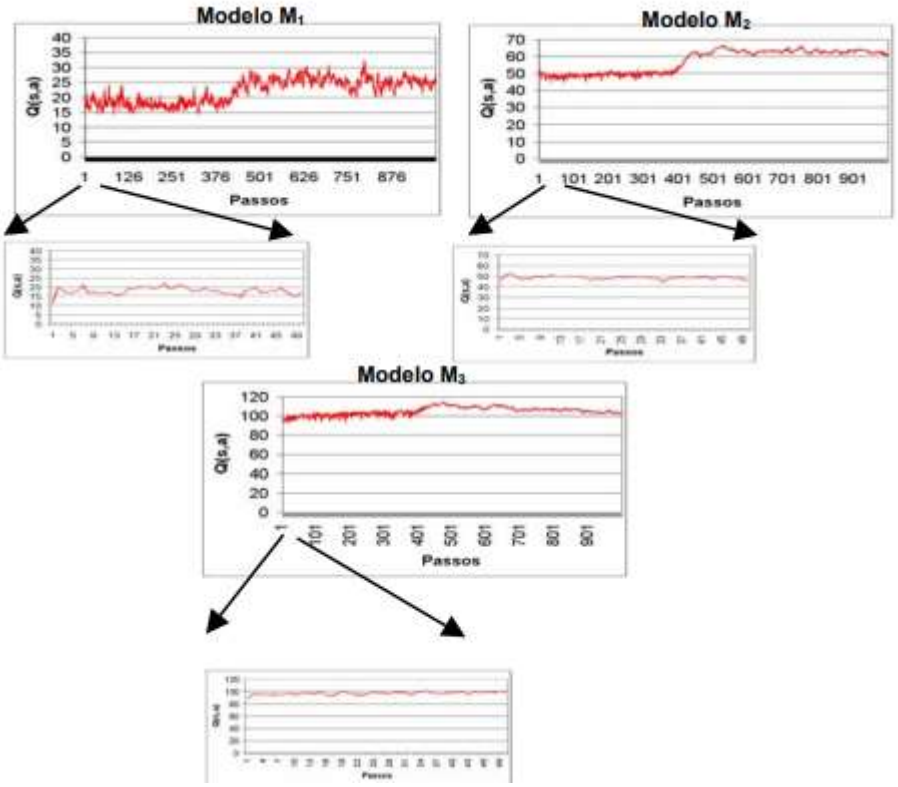


Figura 6. Gráficos dos Q(s, a) dos Modelos M1, M2 e M3.

Na Figura 7, foi usado o mesmo conjunto de simulação da Figura 5 e 6. Os gráficos obtidos representam as médias das transições entre os estados cognitivos dos modelos simulados. Note que o STI não conhece os modelos: os resultados da Figura 7 são conseqüência da convergência do Q-Learning para uma política ótima de ações (μ*).

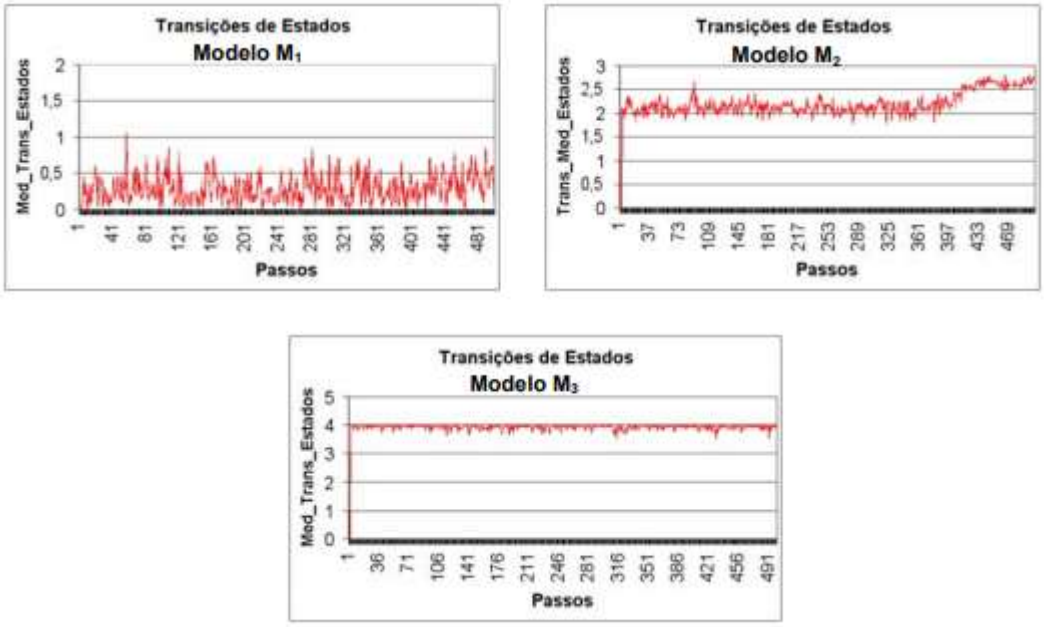


Figura 7. Gráficos da Média das Transições dos Estado dos Modelos M1, M2 e M3.

Na Figura 8, observa-se que o algoritmo Q-Learning consegue através da política ótima de ações (μ), usando os modelos M1, M2 e M3 (que representam os modelos aos quais se esperava obter com as simulações) visitar mais os estados E0 (perfazendo um total de 86% das visitas), E2 (perfazendo um total de 81% das visitas) e E4 (perfazendo um total de 93% das visitas)

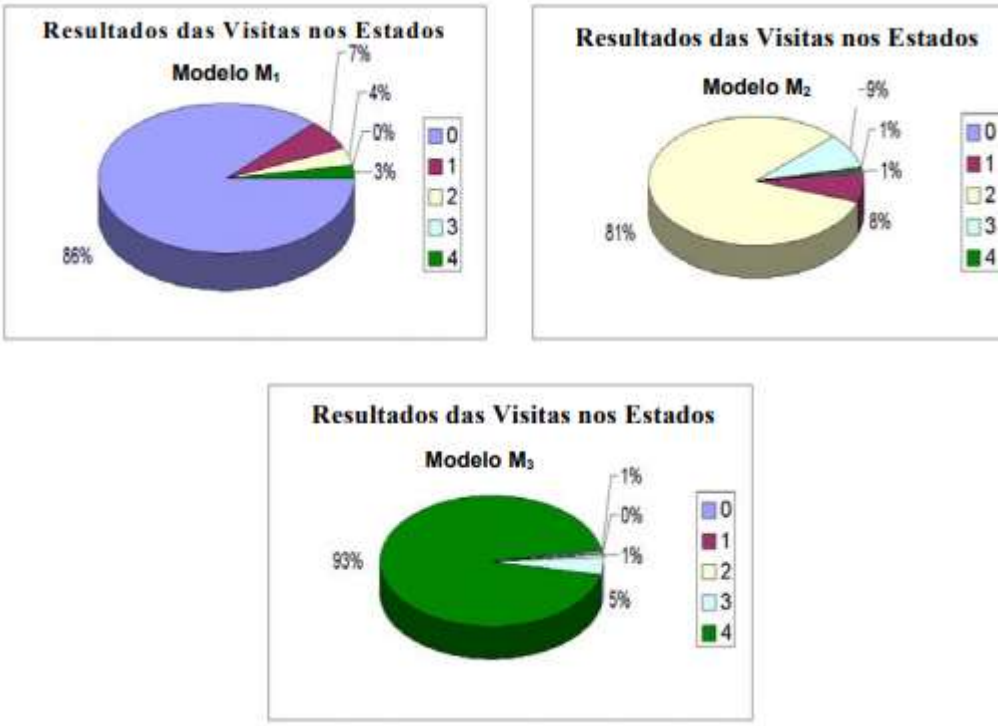


Figura 8. Gráficos das Visitas nos Estados para Modelo M1 M2 e M3.

CONCLUSÃO

Caracterizando o Sistema com uma análise das Vantagens e Desvantagens.

A utilização do Sistema de Aprendizado por Reforço para Modelagem Autônoma do Aprendiz em um Tutor Inteligente pode oferecer vantagens e possíveis dificuldades que se tornariam desvantagens, dentre as quais:

Vantagens:

- Para implementação, não precisa do modelo do aprendiz;

- O sistema independe do conteúdo apresentado pelo tutor;

- Adaptação do sistema a várias estratégias pedagógicas;

- Garantia - através de um log de registro - da continuidade da tutoria do aprendiz a partir do ponto em que parou.

Desvantagens:

- Um número elevado de ações a serem tomadas em determinado estado cognitivo do estudante.

- Dependendo do estado podem existir diversas estratégicas pedagógicas a serem adotadas.

- Interfaceamento com tutores já existentes, devido à necessidade dos tutores receberem os parâmetros do sistema AR.

- Lentidão de convergência do algoritmo, que pode ser parcialmente compensada pelo uso de variantes baseadas em aproximações compactas [Tsitsiklis, J. & Bertsekas, D. 1996], estratégias de generalização da experiência [Ribeiro, C., 1998], ou planos de ação obtidos em simulação [Sutton & Barto, 1998].

ESTUDO DE CASO II

EMÁFORO INTELIGENTE – UMA APLICAÇÃO DE APRENDIZAGEM POR REFORÇO

Introdução

Atualmente, com enorme crescimento do país e apelo comercial automobilístico, nota-se a imensa frota de veículos existente nas ruas das grandes cidades de todo país. Isto faz vivenciar-se com sério problema de mobilidade urbana, problema este que pode ser notado principalmente nos horários de pico, ou seja, ida e volta do trabalho, em que quilômetros de congestionamentos são formados, dado principalmente, por ineficiência no sistema de controle e distribuição do tráfego de veículos. Toma-se como exemplo o fato de ficar parado em um semáforo sendo que na outra via não passa nenhum veículo. Esta ocorrência faz com que o semáforo deixe de atuar como um controlador de tráfego e atue como um intensificador de congestionamento. Foi feito neste projeto um estudo para suprir essa deficiência no controlador de tráfego, com o intuito de projetar um semáforo capaz de tomar decisão de acordo com a situação atual do trânsito, ou seja, semáforo que tome devida decisão em tempo real.

O problema da mobilidade urbana que as grandes cidades vêm enfrentando é citado por Scaringella (2001) em que é enfatizado o uso de tecnologia para controle do tráfego. Para isto estudos da técnica de Aprendizagem por Reforço (AR), sistema estocásticos Markovianos, foram realizados concomitantes com estudos na área de Processo de Decisão Markov (MDP). O estudo para desenvolvimento de semáforo inteligente é tema de ordem científica como demonstrado por Wiering et al (2003), Wiering (2000) e Thorpe (1997), os quais fazem uso do AR.

De acordo com Sutton e Barto (1998) a AR é um formalismo da Inteligência Artificial que permite a um indivíduo aprender a partir da sua interação com o ambiente no qual ele está inserido. A aprendizagem se dá através do conhecimento sobre o estado do indivíduo no ambiente, das ações efetuadas e das mudanças de estado decorrentes das ações que são elementos essenciais na área de AR, a qual vem sendo muito utilizada com sucesso em problemas reais nos últimos anos (Kaelbling, Littman, e Moore, 1996). Este trabalho apresenta um problema de aprendizado dos tempos envolvidos em um cruzamento contendo dois semáforos, de modo que o haja a maximização do

somatório de carros em ambas as vias. Foram utilizados, o software matemático MatLab® e seu pacote de simulação SimEvents®, que produz resultados através de equações matemáticas e que tem como objetivo proporcionar melhores soluções para o desenvolvimento do processo. O algoritmo de aprendizagem por reforço utilizado neste problema foi o SARSA (Singh, Jaakkola, Littman e Szepesvári, 2000), o qual é bem aplicado em sistemas com aprendizagem em tempo real

O artigo está organizado como se segue: primeiramente é apresentado a teoria de Processo Decisório de Markov (MDP), o qual é a base teórica para AR, a qual é apresentada logo em seguida; apresentasse-se então a modelagem do problema com o resultados na seção a seguir; finalizando o artigo são apresentadas conclusões e as perspectivas de trabalhos futuros.

Processo Decisório de Markov

Um Processo Decisório de Markov (MDP) é uma forma de modelar processos onde as transições entre estados são probabilísticas, os estados são observáveis e é possível interferir com o sistema dinâmico através de ações que produzem mudanças de estado e recompensas. Cada ação tem uma recompensa (ou custo), que depende do estado em que o processo se encontra. Este processo é ditos “de Markov” (ou “Markovianos”) porque os processos modelados obedecem á propriedade de Markov: o efeito de uma ação em um estado depende apenas da ação e do estado atual do sistema (e não de como o processo chegou a tal estado); e são chamados de processos “de decisão” porque modelam a possibilidade de um agente (ou “tomador de decisões”) interferir periodicamente no sistema executando ações. MDPs podem ser aplicados em um grande número de áreas diferentes, por exemplo, finanças e investimentos, inspeção, manutenção e reparação, recursos hídricos, como mostra White (1993).

Definição MDP

Um Processo de Decisão de Markov (MDP) é uma tupla (S, A, T, R), onde:

• S é um conjunto de estados em que o processo pode estar;

• A é um conjunto de ações que podem ser executadas em diferentes épocas de decisão;

• T: S x A x S → [0, 1] é uma função que dá a probabilidade de o sistema passar para um estado s" S, dado que o processo estava em um estado s S e o agente decidiu executar uma ação a A (denotada T(s"| s, a)); e

• R: S x A → R é uma função que dá o custo (ou recompensa) por tomar uma decisão A quando o processo está em um estado S.

Considerando que o sistema está em algum estado s e em dada época decisão k, é necessário selecionar qual ação a deve ser executada. A ação é selecionada seguindo uma regra de decisão, sendo uma forma simples de regra de decisão o mapeamento direto de estados em ações. O conjunto de todas as regras de decisão é chamado de política (π). Ao executar uma política, o tomador de decisões receberá recompensas em cada época de decisão. Para comparar duas políticas, é necessário um critério de desempenho que pode ser definido por diversos critérios de desempenho (ou “de otimalidade”) para MDPs, e entre os mais conhecidos pode-se citar:

• A recompensa média por época de decisão:

$$\frac{1}{z}\sum_{k=0}^{z-1} r$$

• A recompensa esperada total, E :

$$\sum_{k=0}^{z-1} r$$

• A recompensa esperada descontada, E:

$$\sum_{k=0}^{z-1} \gamma^{k} r_{k}$$

Sendo z o horizonte de um MDP. Neste trabalho foi determinado que a recompensa fosse a diferença da quantidade de carros de passaram pelo

semáforo e os que ficaram na fila. Para isto foi utilizada a recompensa média por época das amostras coletadas em tempos pré-definidos. Portanto um estado ótimo para este modelo é quando se tem a máxima recompensa, ou seja, quantidade máxima de carros que passaram pelo menos a quantidade de carros que ficaram na fila. O fator de desconto γ]0,1[é usado com horizonte finito para garantir a convergência do valor da recompensa total esperada. Uma política é ótima (π*) quando a recompensa total esperada para todo estado é maximizada. A função valor V*(s) dá a recompensa total esperada para uma política ótima π*:

$$V^*(s) = max_{a \in A}[R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a)V^*(s')] \quad (4)$$

Dado um estado s ϵ S, uma ação a ϵ A e uma política π para um MDP, pode-se definir o valor da ação a no estado s, considerando a recompensa imediata de a e a recompensa esperada após a, nas outras épocas de decisão, desde que as ações tomadas após a sejam determinadas pela política π. A função que dá este valor é denotada por Q. Para a esperança da recompensa total descontada, Q é definida como:

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s \in S} T(s'|s,a)V^{\pi}(s') \quad (5)$$

Para uma política ótima π*, tem-se:

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s \in S} T(s'|s,a)V^*(s')$$

$$\pi^*(s) = argmax_{a \in A} Q^*(s,a)$$

$$V^*(s) = max_{a \in A} Q^*(s,a) \quad (6)$$

Aprendizagem por Reforço

Aprendizado por Reforço (AR) é uma técnica de aprendizado de máquina, bastante usada em controle de processos industriais, em que um agente aprende por sucessivas interações com o seu ambiente e escolhe as ações que proporcionam os melhores resultados/ganhos. O ambiente apresenta, a cada interação, um novo estado (situação) e um valor numérico chamado reforço para avaliar a ação (conforme Figura 1).

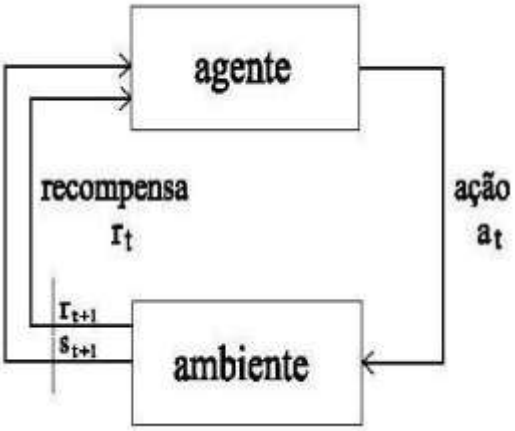


Figura 1 - Interação entre agente e ambiente

O domínio deve ser modelado como um MDP, onde o agente e o ambiente interagem em uma seqüência discreta de passos no tempo, o estado e a ação em dado instante determinam a distribuição de probabilidades para o estado seguinte e o reforço. O objetivo do agente normalmente é escolher ações de modo a maximizar uma soma descontada dos reforços subseqüentes

A principal diferença entre o Aprendizado por Reforço e outras técnicas de aprendizado de máquina é a utilização da avaliação das ações tomadas (Sutton e Barto, 1998). Em outros métodos, são utilizados instruções ou exemplos, informando as situações e as ações corretas que devem ser tomadas. Na AR, o agente tenta descobrir, dentre as possíveis ações, quais delas promovem melhores resultados, utilizando apenas sua própria experiência. A Função de Recompensa define, para o estado atual, qual a melhor ação imediata enquanto a Função de Valor permite a avaliação das possíveis ações em longo prazo. Estas funções não são alteradas durante as interações enquanto são utilizadas para atualizar a Política, definindo os melhores mapeamentos estado-ação.

Características da aprendizagem por reforço

Os elementos que caracterizam Aprendizagem por Reforço são:

• Aprendizado por interação: característica principal que define AR. O agente AR age no ambiente e aguarda pelo valor do reforço/recompensa que o

ambiente retorna em resposta perante á ação tomada, assimilando através do aprendizado o valor obtido para tomar decisões posteriores.

• Retorno atrasado: um valor de reforço alto enviado pelo ambiente ao agente não significa necessariamente que a ação tomada por este é recomendada. Uma ação é produto de uma decisão local no ambiente, sendo seu efeito imediato de natureza local, enquanto que em um sistema de AR o intuito é alcançar objetivos globais. Ou seja, a qualidade das ações é vista pelas soluções em longo prazo.

• Orientado pelo objetivo: em AR não é necessário conhecer detalhes da modelagem do ambiente. Simplesmente existe uma gente que age neste • Investigação versus exploração: esta questão consiste em decidir quando se deve aprender e quando não se deve aprender sobre o ambiente, mas usar a informação já obtida até o momento. Para que um sistema seja realmente autônomo, esta decisão deve ser tomada por ele próprio. A decisão é fundamentalmente uma escolha entre agir baseado na melhor informação que o agente dispõe no momento ou agir para obter novas informações sobre o ambiente que possam permitir níveis de desempenho ainda maiores no futuro. Em suma o agente deve aprender quais ações maximizam os ganhos obtidos, mas também deve agir de forma a atingir esta maximização explorando ações ainda não executadas ou regiões pouco visitadas no espaço de estados. Uma boa estratégia então é mesclar os modos de investigação e exploração. ambiente desconhecido tentando alcançar um objetivo, que geralmente é aperfeiçoar algum dentro do ambiente.

No algoritmo SARSA há possibilidade de se utilizar vários tipos de políticas de aprendizagem. A utilizada neste modelo foi o GLIE (Greedy in the Limit with Infinite Exploration). Estas políticas de aprendizado podem ser expressas a partir de probabilidades e um exemplo da política GLIE é uma das formas de exploração de Boltzman.

$$\Pr(a|s,t,Q) = \frac{e^{\beta t(s) Q(s,a)}}{\sum_{b \in A} e^{\beta t(s)\, Q(s,b)}}$$

Onde β(t) é o coeficiente de exploração do tempo t que controla a taxa de exploração na política de aprendizado, isto é, quanto mais β(t) tende ao infinito, mais guloso o sistema se torna, convergindo mais rapidamente, explorando mais e investigando menos. Para β(t) com valores baixos o sistema permanece em maior fase de investigação do que exploração, isto gera uma convergência mais próxima do ponto ótimo, mas com tempo de convergência maior.

Modelagem do Problema

O cruzamento que se deseja modelar pode ser representado pela Figura 2, ou seja, um cruzamento e dois semáforos.

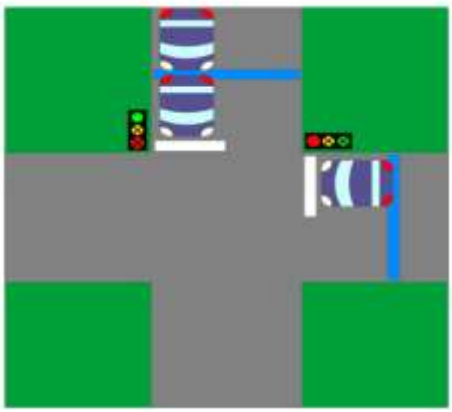


Figura 2 – Cruzamento com dois semáforos.

Neste trabalho o estado foi definido como a tupla: (Período, Largura do Pulso). Cada estado possui um período e uma porcentagem (largura de pulso) que corresponde à porcentagem do período em que um semáforo fica aberto e o outro fechado. Esta variação da largura de pulso pode ser observada nas Figuras 3 e 4:

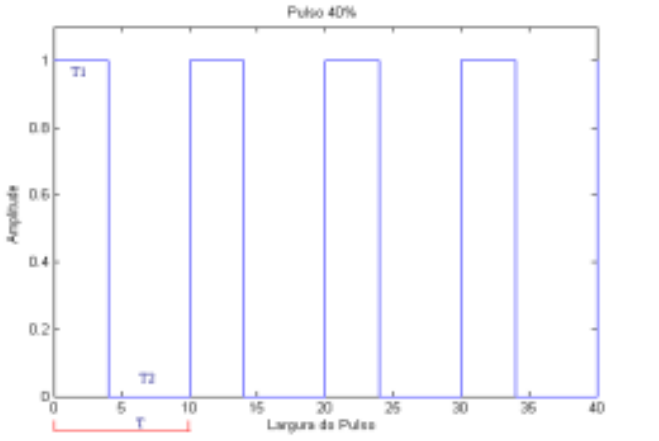


Figura 3 – Largura de pulso 40%

Onde T = T1+T2; Por exemplo, para T=10s e uma largura de pulso de 40% o tempo que o semáforo #1 fica aberto é dado pelo tempo T1 e o tempo que o semáforo #2 fica aberto é dado por T2, como pode ser observado na figura 3. Na figura 4 é exemplificada a largura de pulso de 80%:

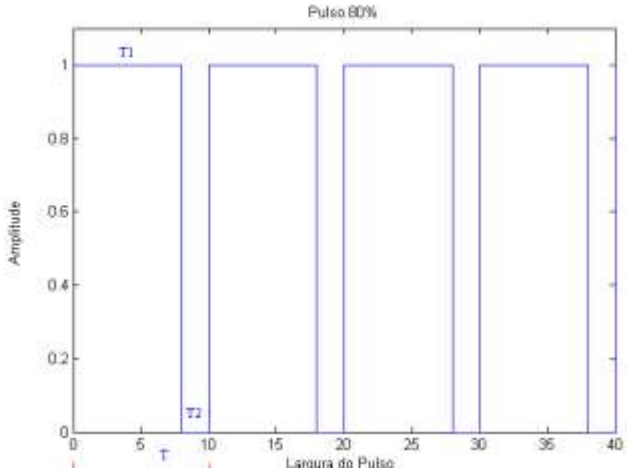


Figura 4 – Largura de pulso 80%

Nota-se que o tempo que o semáforo #1 fica aberto é maior que o tempo em que o semáforo 2 fica aberto, por ter usado uma largura de pulso de 80%. O algoritmo de aprendizagem por reforço tem a função de encontrar, a partir de iterações, este estado ótimo, isto é, o melhor período e respectiva largura de pulso. A recompensa foi determinada como sendo a diferença da quantidade de carros de passaram pelo semáforo e os que ficaram na fila, como descrito na equação 10.

RM= (X(tamanho(X,1)-X(1)))/tamanho(X,1)- (média(X1)+ média(X2))

Onde X é a amostra da quantidade de carros que passaram pelo cruzamento no tempo de simulação estipulado, X1 é a quantidade de carros da fila1 que passaram pelo cruzamento, e X2 é a quantidade de carros da fila 2 que passaram pelo cruzamento. Para isto foi utilizada a recompensa média por época das amostras coletadas em tempos pré-definidos.

Portanto um estado ótimo para este modelo é quando se tem a máxima recompensa, ou seja, quantidade máxima de carros que passaram pelo cruzamento menos a quantidade de carros que ficaram na fila. Fazendo análise dos métodos de convergências, concluímos que para o projeto em desenvolvimento, o melhor método a ser usado é o SARSA.

Pois se trata da simulação de um cruzamento que pode apresentar diversos estados, isto é, diferentes períodos e largura de pulso. Com esta busca constante pela melhor ação a ser tomada a cada estado, este método permite encontrar um funcionamento ótimo dos semáforos nos cruzamentos, evitando geração de congestionamento muitas vezes desnecessários. A modelagem foi feita a partir do esquemático feito no SimEvents®, pacote MatLab®, a qual é apresentada na Figura 5.

Resultados

Os resultados a seguir foram coletados a partir da simulação do modelo desenvolvido. Foram feitas varias simulações que poderão ser observadas nos gráficos abaixo. Primeiramente, nas Figuras 6 e 7 pode ser observado um exemplo de fila gerada por um bloco específico do software utilizado, que pode representar um fila de trânsito real.

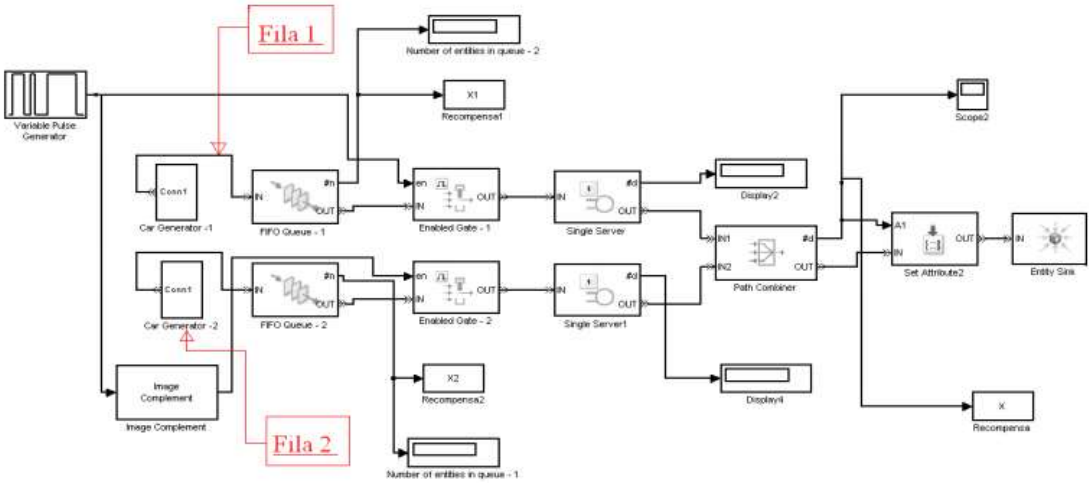


Figura 5 – Modelo de um cruzamento SimEvents® Inicialmente será simulado um Período de 50 segundos e largura de pulso igual 20%, isto é, o semáforo #1 fica aberto 10 segundos enquanto que o semáforo #2 fica aberto 40 segundos. Os gráficos das entidades nas filas podem ser observados nas figuras seguintes, bem como o gráfico que representa as entidades que passaram pelo bloco, para o estado específico.

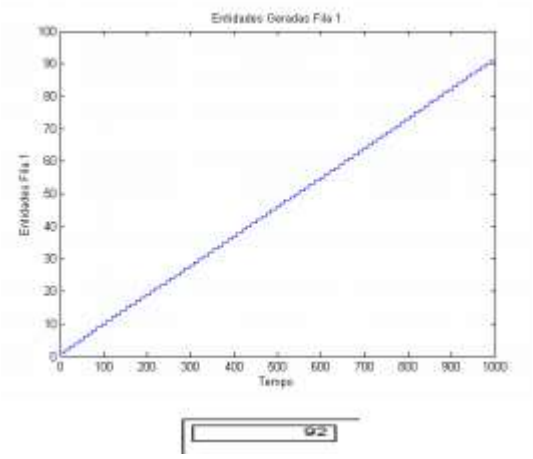


Figura 6 – Número de entidades na Fila 1

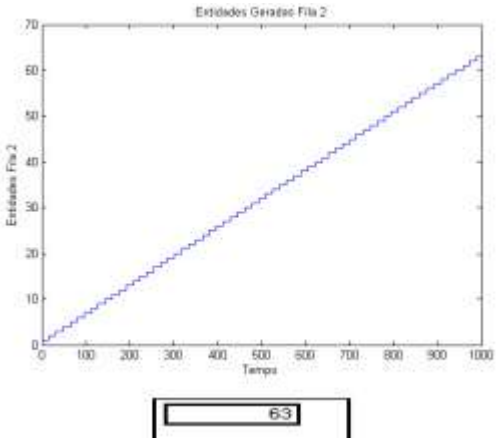


Figura 7 – Número de entidades na Fila 2

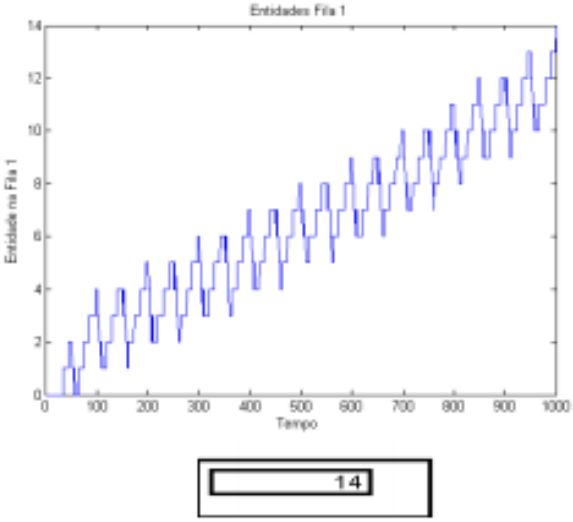


Figura 8 – Número entidades na Fila 1

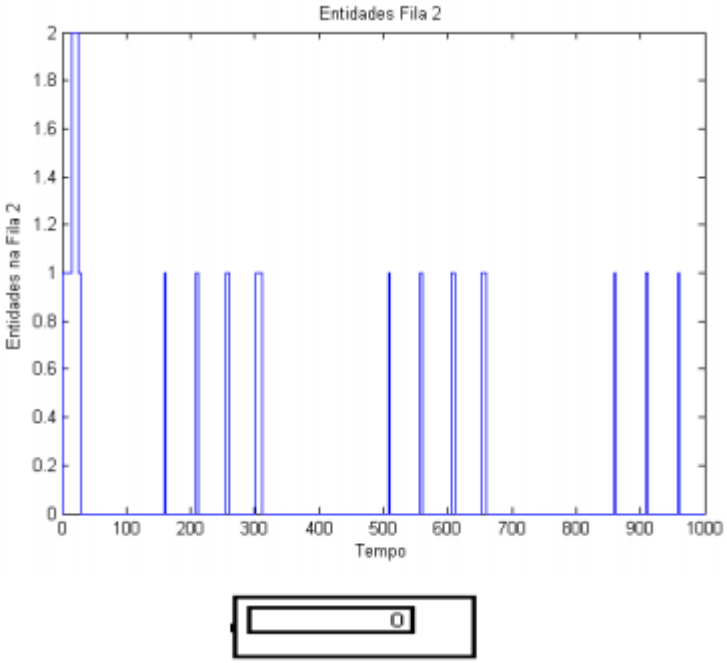


Figura 9 – Número entidades na Fila 2

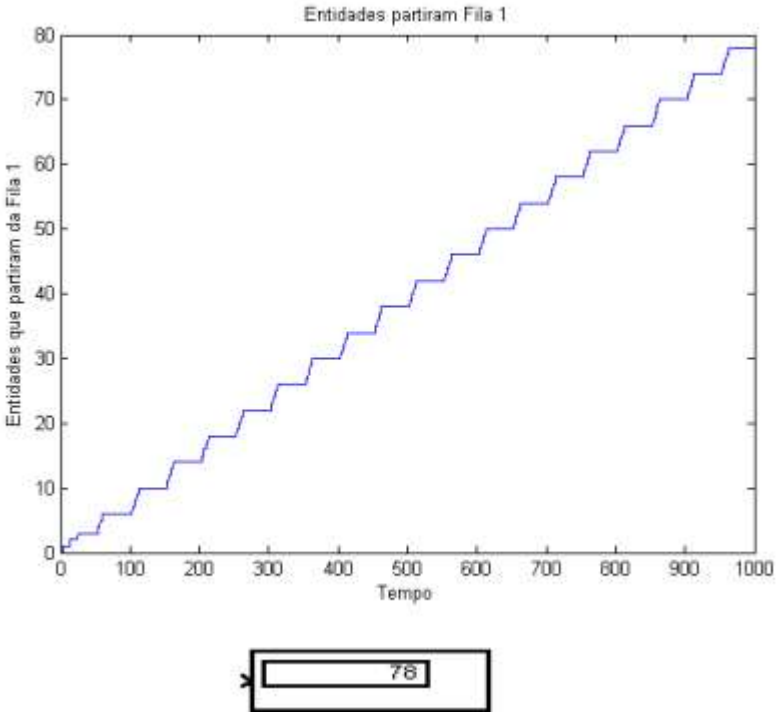


Figura 10 – Número de entidades da fila 1 que já passaram o cruzamento

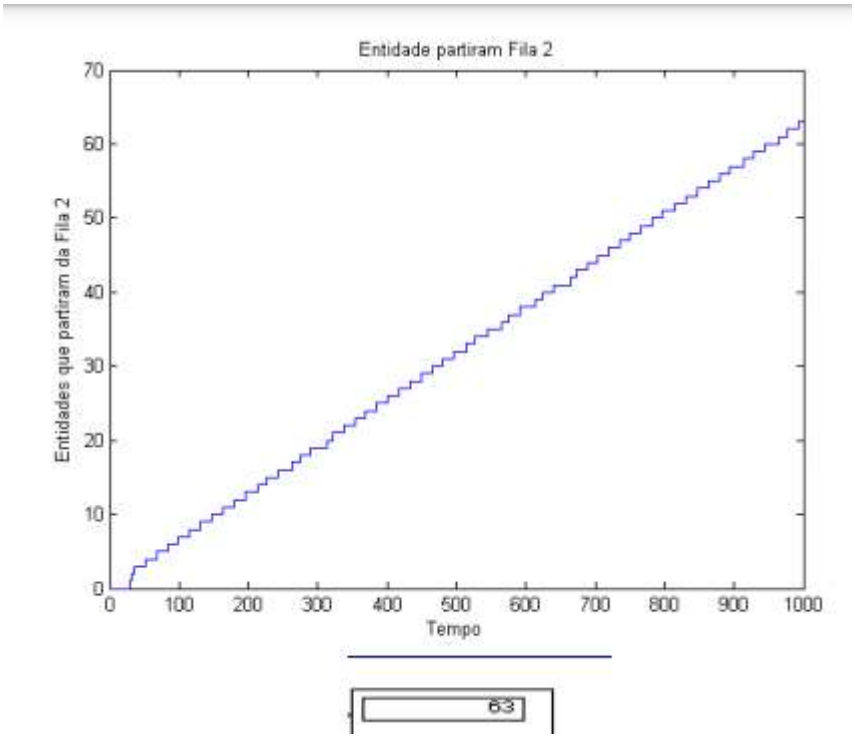


Figura 11 – Número de entidades da fila 2 que passaram o cruzamento

Como descrito anteriormente, a recompensa média será calculada a partir da diferença da quantidade de carro que passaram pelo semáforo e a quantidade de carros que ficaram na fila, equação 10. Fazendo este cálculo para este estado aleatório, obteve-se RM= 1,9412. Pode-se observar que o cálculo do tempo ótimo envolvido na abertura e fechamento dos semáforos e a máxima recompensam não são triviais, pois devem ser levados em considerações diversos estados possíveis de atuação. Para isso será utilizado como princípio técnica de Aprendizagem por Reforço, onde se espera que o agente encontre a melhor ação a ser tomada em um estado qualquer. Esta integração do AR com o cálculo dos tempos e porcentagem foi desenvolvido utilizando a área de programação do MatLab® e fazendo a conexão com o modelo desenvolvido no Simulink.

Para se ter como base de tempo o período foi limitado para variar entre 30 e 90 segundos, já a largura do pulso pode ser de 1% a 99% do período. A execução de programas de Aprendizagem por Reforço requer certo número de episódios, sendo que em cada episódio é sorteado um estado qualquer e partir deste busca um estado ótimo. O episódio evita também que o programa fique

„preso" em 2 ou mais estados, e ainda cada episódio contêm vários passos que o programa executa para encontrar um estado ótimo para aquele episódio. Neste programa foram feitas simulações usando 10 episódios e cada episódio contendo 200 passos. Como explicado anteriormente, o valor de β(t) utilizado inicialmente foi de 1, ou seja, um valor baixo considerado que a cada passo é acrescido de um valor pré-definido.

Com este valor inicial de β(t) o sistema estava em época de maior investigação do que exploração, de acordo que as iterações foram passando β(t) aumentava e então o sistema passava a fase de exploração. Será representado inicialmente cada episódio e seu estado ótimo, posteriormente serão mostrados todos os episódios em um só gráfico. O 1º Episódio, 200 passos, está representado abaixo, o estado inicial foi sorteado aleatoriamente e a partir dele se encontrou o estado ótimo para este episódio.

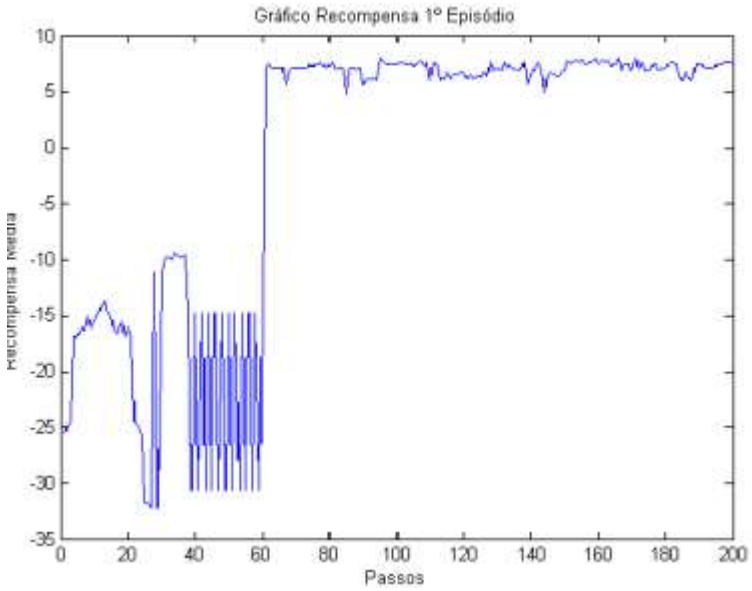
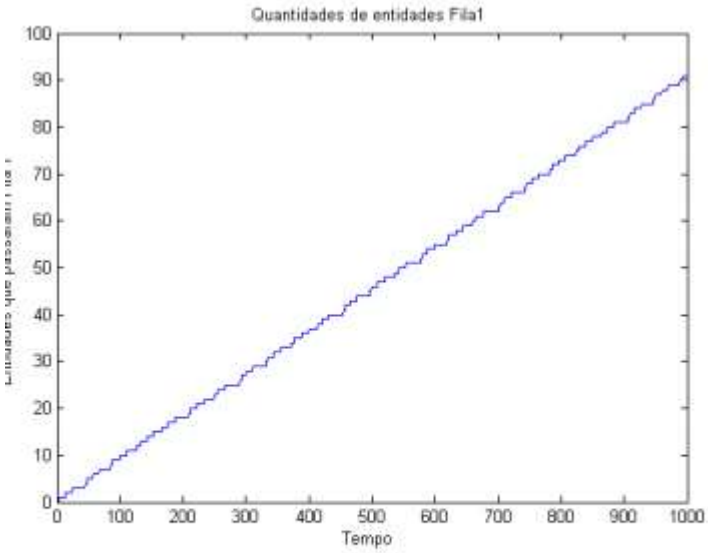


Figura 12 - Gráfico de Recompensa Média - 1º. Episódio

Este 1º. episódio começou suas iterações a partir do estado 50 segundos e 45% de largura de pulso. A partir deste estado, teve-se 200 passos e conclui-se que o estado ótimo para este episódio é 41 segundos e 39% de largura de pulso. Fazendo este cálculo para este estado, utilizando a equação 10, obteve-se RM = 8,0588. Este estado ótimo representa uma passagem máxima de entidades pelo semáforo, tendo consequentemente fila mínima. Isto pode ser

comprovado comparando os gráfico das Figuras 13 e 14 de entidades que passaram pelo cruzamento e o gráfico das Figuras 6 e 7 que representa a quantidade de entidades geradas.

No 7º. episódio as iterações começaram a partir do estado 58 segundos e 33% de largura de pulso. A partir deste estado, teve-se 200 passos e conclui-se que o estado ótimo para este episódio é 38 segundos e 68% de largura de pulso. Fazendo o cálculo para este estado, utilizando a equação 10, obteve-se RM = 7. Podem-se observar os gráfico das figuras 16 e 17 de entidades que passaram pelo cruzamento e o gráfico das figuras 6 e 7 que representa a quantidade de entidades geradas.



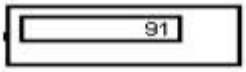


Figura 13- Quantidade entidades da fila 1 que passaram o cruzamento.

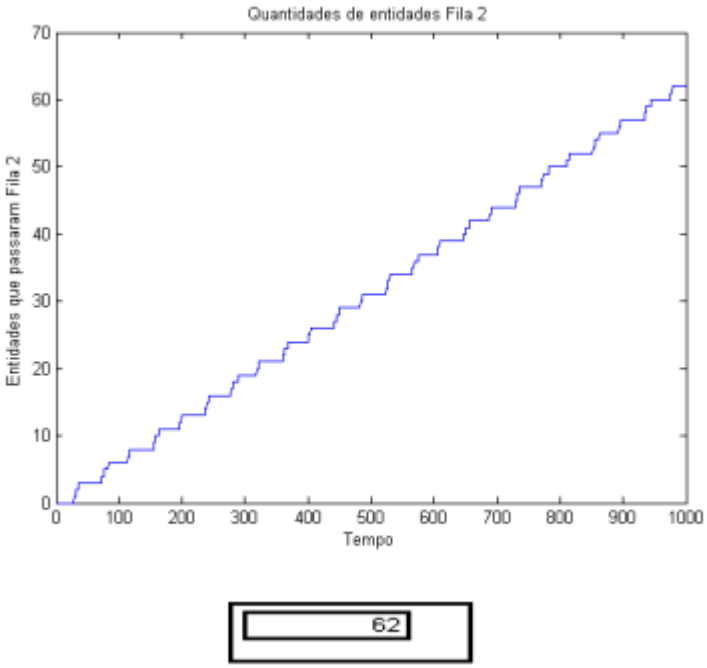


Figura 14- Quantidade entidades da fila 2 que passaram o cruzamento.

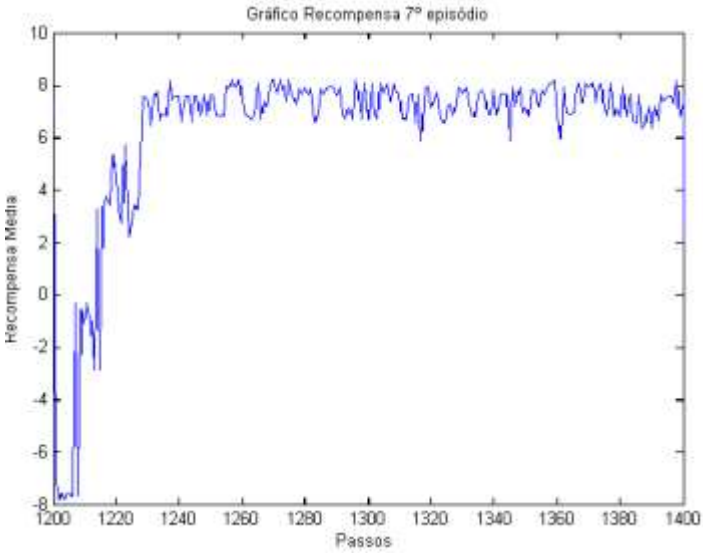


Figura 15 - Gráfico de Recompensa Média- 7º Episódio

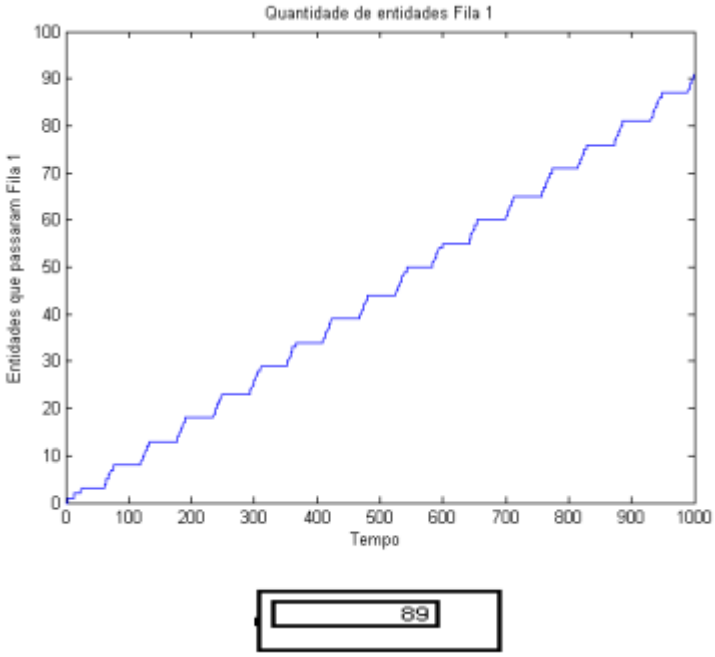


Figura 16- Quantidade entidades da fila 1 que passaram o cruzamento.

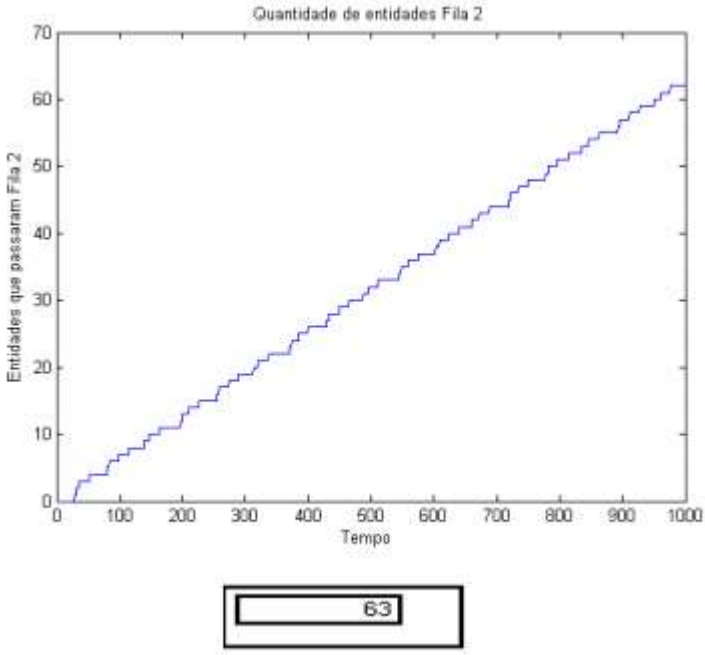


Figura 17- Quantidade entidades da fila 2 que passaram o cruzamento.

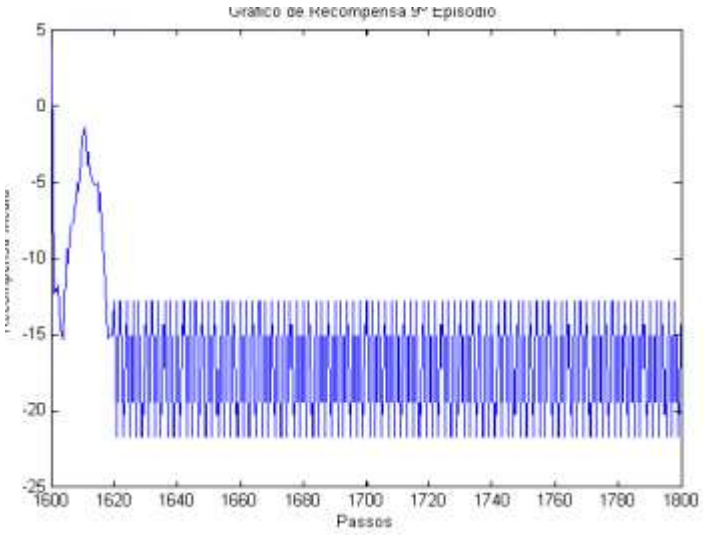


Figura 18 - Gráfico de Recompensa Média - 9º Episódio

Neste 9º. episódio, como pode ser observado na Figura 18, ocorreu de o sistema ficar preso entre os estados 53 segundos, 99% e 30 segundos, 2%, não visitando outros estados. Isto se dá por ser um sistema de aprendizagem, no decorrer das iterações isso se torna mais difícil de acontecer.

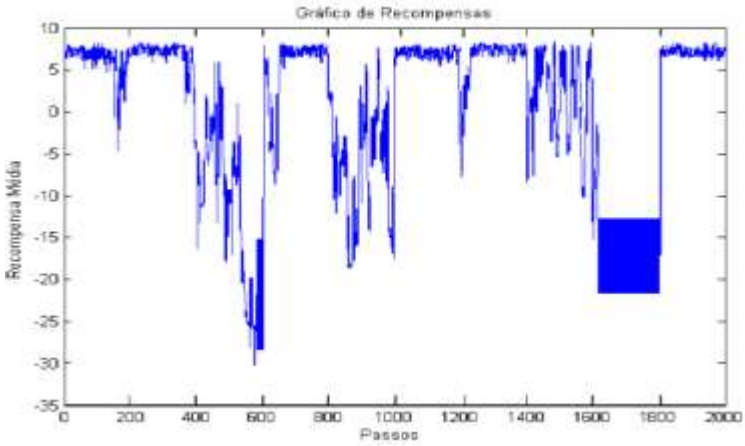


Figura 19 - Gráfico de Recompensa Média

Neste gráfico estão representados todos os episódios e passos desta simulação. Pode-se notar que a cada novo episódio o sistema tende a encontrar um novo estado ótimo até que a recompensa seja máxima. Para este sistema o estado ótimo é com um período de 30 segundos e largura de pulso 88%. Neste estado, pode-se notar nas figuras 20 e 21 que a todos os carros passaram pelo

cruzamento, tendo assim fila mínima. Calculando a recompensa para este estado, utilizando a equação 10, obteve-se RM= 8,4118.

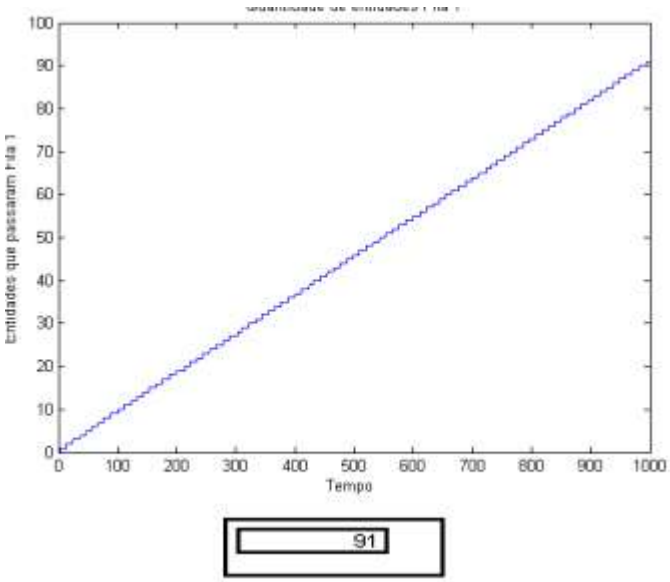


Figura 20- Quantidade entidades da fila 1 que passaram o cruzamento.

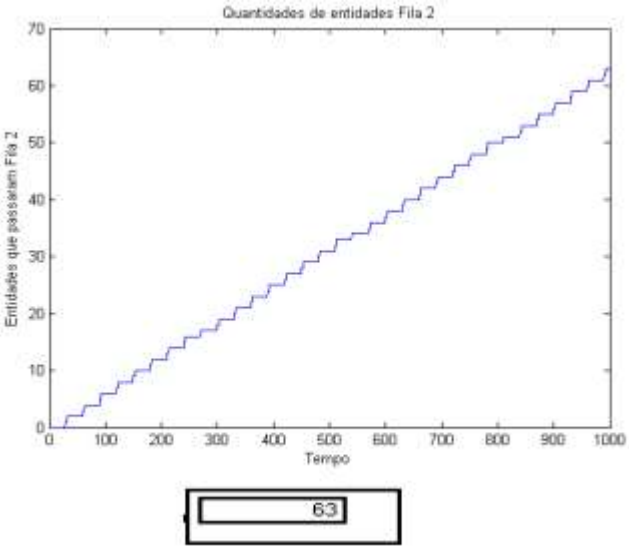


Figura 21- Quantidade entidades da fila 2 que passaram o cruzamento.

Conclusões e Trabalhos Futuros

omo pode ser visto nos gráficos apresentados, quando foi feita a simulação do sistema considerando apenas um estado qualquer, sem aplicação da Aprendizagem por Reforço, obteve-se uma recompensa média muito inferior ao estado obtido pela aplicação da técnica Aprendizagem por Reforço.

Trabalhos como esse são válidos para o desenvolvimento tecnológico, uma vez que relacionam os conceitos presentes no meio acadêmico com processos reais presentes em indústrias, ou seja, essa é uma forma de se aplicar a teoria à prática obtendo resultados proveitosos. Nesse projeto foi elaborada uma forma de integrar o software SimEvents®, com a técnica de Aprendizagem por Reforço a fim de gerar um sistema que opere continuamente em estado ótimo. Este projeto servirá como base de próximos trabalhos que estão sendo desenvolvido, considerando neste, quatro cruzamentos e oito semáforos interdependentes e dependência no tempo, isto é, o fluxo de carros depende da instante atual e tem picos, como por exemplo na hora do rush.

REFERÊNCIAS

Burns,H and Parlett,J.W. and Redfield,C.L.: “Intelligent Tutoring Systems” N.J.: Lawrence Erlbaum-Verlag 1991.Bellman, R. E.

Dynamic Programming. Princeton: Princeton University Press, 1957.

Giraffa, L.M.M.: “Uma arquitetura de tutor utilizando estados mentais”, Rio Grande do Sul: PUC-RS,1999. Tese de Doutorado.

Hendley, R. and Jurascheck , N. “Cascade: Introducing AI intoCBT.Computers Education”, London, v.18, n.1-3, p.71-76, 1992

Leite, A. S. “Um Modelo de Sistema Educativo Cognitivista Baseado em Tutoria Inteligente Adaptativa via Aderência Conceitual”, São José dos Campos: ITA, 1999 Tese de Doutorado.

Marietto, M. G.:”Definição Dinâmica de Estratégias Instrucionais em sistema de tutoria Inteligente: Um Abordagem Multiagentes na WWW”, São Jose dos Campos:ITA, 2000

Tese de Doutorado. Sutton, R.and Barto, A. Reinforcement Learning: An Introduction. MIT Press, 1998. Ribeiro, C.H.C. “Embedding a priori Knowledge in Reinforcement Learning”, Journal of Intelligent and Robotic Systems. Dordrecht, Holanda: , v.2, n.1, p.51 - 71, 1998

Viccari, R.M.: “Um Tutor Inteligente para a Programação em Lógica – Idealização,Projeto e Desenvolvimento”,Portugal Universidade de Coimbra,1990,Tese de Doutorado. Tsitsiklis, J. e Bertsekas, D. (1996).

Neuro-Dynamic Programming. Athena Scientific, 1996. Watkins, C. J. C. H.:” Learning from Delayed Rewards.”, PhD thesis, University of Cambridge,1989.

Watkins, C. J.C.H., and Dayan, P. “Q-leaning”. Machine Learning 8(3/4):279-292, 1992

Scaringella, R.S.(2001) A crise da mobilidade urbana em São Paulo. São Paulo Perpec. [online], vol.15(1).

Wiering, M., Van Veenen , J., Vreeken, J. & Koopman, A. (2003) Intelligent Trafic Light Control. European Research Consortium for Informatics and Mathematics, vol. 53, pp. 40-41. Wiering, MA. (2000)

Multi-agent reinforcement learning for traffic light control. Proceedings of the Seventeenth International Conference on Machine Learning (ICML2000), pp. 1151-1158. Thorpe, T.L. (1997)

Vehicle Traffic Light Control Using Sarsa. Master"s thesis. Department of Computer Science, Colorado State University. Sutton, R. S. & Barto, A.G.

(1998) Reinforcement Learning: An Introduction. The MIT press. Kaelbling, L. P., Littman, M. L. & Moore, A. W. (1996)

Reinforcement Learning: A survey. Arxiv preprint Singh, S.; Jaakkola, T.; Littman, M. L and Szepesvári, C (2000). Convergence results for single-step on-policy reinforcement learning algorithms. Machine Learning, Vol. 38, No. 3, pp. 287-308 White, D. J. (1993)

A survey of applications of Markov decision processes. The Journal of the Operational Research Society, v. 44, n. 11, p. 1073–1096.